Artificial Intelligence and Robotics

A Complete Course in Plain English

ARTIFICIAL INTELLIGENCE AND ROBOTICS

Volume Four

Computer Vision

TBUGE MECHATRONIX

Artificial Intelligence and Robotics, Volume Four: Computer Vision

First edition. TBUGE Mechatronix.

This volume assumes that a camera is a sensor and that a picture is a measurement. It teaches vision from the pixel upwards: how light becomes numbers, how those numbers are filtered, how structure is found in them, and how a system is built that keeps working when the light changes.

The programme runs to forty-eight volumes. Volume Three taught a machine to improve from examples; this one gives it eyes, and the judgement to know when its eyes are lying.

Course codes have been left out on purpose. A subject should be named, not numbered.

Contents

Preface **6**

How This Volume Is Arranged **7**

Part One · The Image as Evidence **9**

How a Camera Becomes a Number 10

Pixels, Coordinates and the Cost of Sampling 16

Convolution: The One Operation Everything Uses 22

Part Two · Structure Without Learning **27**

Edges, Gradients and the Honest Line 28

Corners, Features, and the Points That Survive 33

Segmentation: Regions, Thresholds and the Shape of Things 39

Depth: Two Eyes, One Camera, and the Arithmetic of Distance 44

Part Three · Learning to See **49**

Learning to See 50

Training a Vision Model Without Fooling Yourself 55

Detection and Measurement 60

Tracking Across Frames 65

Part Four · Making It Work **70**

Light, Lenses and the Rig 71

When Vision Is the Wrong Tool 77

Glossary **82**

Sources and Further Reading **85**

Preface

Vision is the sense everyone assumes a machine already has. We point a camera at a part, the software finds the defect, and the transaction looks trivial because human beings do it without effort twenty times a second. Underneath that effortlessness sits a chain of assumptions — about light, about focus, about what a defect looks like, about what "looks like" even means when the surface is scratched, oily, or moving.

When the chain breaks, it breaks quietly. A camera does not complain that the exposure has doubled since the last service. It does not mention that the new supplier's parts are two shades darker. It reports a confidence score, and the score is confident. Vision failures are almost always failures of the physical setup being described as failures of the model.

So this volume begins before the model. Chapters on sensors, sampling, optics and light come first, because the arithmetic of a lens determines what any algorithm downstream can possibly see. The middle chapters build the classical tools: convolution, edges, features, thresholding, depth. They remain in daily use in factories, and more importantly they teach the vocabulary that modern networks inherited — the kernel, the receptive field, the response map. The later chapters turn to learning, and to the unglamorous work that decides whether a vision system survives its second month: calibration, dataset capture, honest splitting, tracking, and the maintenance of a rig that drifts by a fraction of a millimetre a week.

What this volume asks of you

Arithmetic, and a willingness to draw. Every operation in the first half of the book can be done with a pencil on a grid of nine numbers, and doing it once is worth reading about it ten times. When a kernel is introduced, the page shows a small patch of pixels and the sum that produces one output value. Work it through on paper. The moment the mechanic becomes obvious is the moment the rest of the subject becomes tractable.

A camera helps. Even a phone camera and a torch will do for most of the exercises: photograph a textured surface at an angle, photograph a printed edge,

photograph the same object twice under different light, and look at the numbers.

What it does not ask

No calculus, no matrix algebra, no framework. Networks appear near the end, and they are described in terms of the same convolution already built by hand, so that a reader who can compute a Sobel response has the concept needed to understand a layer.

A word about the pictures

A book about images has an obvious difficulty: the pictures live in the reader's head and in the code, not on the page. Where a figure would help, the text gives a small table of numbers instead, and describes what you would see. It is slower than a photograph and it is more honest, because you can check the numbers.

The promise

By the last chapter you should be able to stand in front of a machine, look at the camera, the light and the part, and estimate — before any software runs — whether the task is possible, how many pixels it needs, how much light it needs, and where it will fail. That estimate is the difference between a project that ships and a project that produces a demonstration.

How This Volume Is Arranged

The book has four parts, and they correspond to four questions a vision engineer asks in order.

Part One — The Image as Evidence is about the measurement. A camera converts light into integers, at a particular resolution, over a particular field of view, with a particular amount of noise. Those decisions are made before any algorithm runs, and they cap everything that follows. Three chapters: how light becomes a number, how sampling fixes what can be resolved, and how a kernel transforms a neighbourhood into a single value.

Part Two — Structure Without Learning is the classical toolkit, and it is still the right first attempt on most industrial problems. Edges, corners, thresholds, regions, depth. Each idea is worked on a nine-number patch so that the arithmetic is visible, and each is followed by the conditions under which it collapses: glare, occlusion, repetitive texture, a change of scale.

Part Three — Learning to See connects the classical vocabulary to convolutional networks, then spends its longest chapter on the part that decides whether a model is real: splitting a dataset so that the test set has not been contaminated by the training set. Detection, measurement uncertainty and tracking close the part, because a vision system in motion is a different problem from one that inspects still frames.

Part Four — Making It Work is engineering. Optics and lighting arithmetic, calibration, depth of field, motion blur, and the cost model that tells you when a laser, a load cell or a limit switch is a better answer than a camera.

The chapters are not identical in shape. Some are built around a single worked calculation, some around a failure story, one is a checklist. A book that repeats the same five sections thirteen times teaches the reader to skip, and the habit of skipping is fatal in a subject where the detail is the whole thing.

Every chapter closes with work. The headings vary — Problems, Exercises, Things to Try, Working Notes — because the work varies: some chapters want arithmetic, some want a decision, some want you to go and look at something with a camera.

PART ONE

The Image as Evidence

CHAPTER ONE

How a Camera Becomes a Number

The complaint arrived on a Tuesday, from a customer who had bought four hundred brackets. Three of them would not fit. The inspection cell had passed all four hundred, and its report was clean: no defects found, confidence high.

The engineer who investigated did the obvious things first. She re-ran the images through the model. She checked the model version against the one that had been validated. She looked at the rejected brackets on the bench, and then at the photograph the cell had taken of each one, and the answer was sitting there in plain sight. In the photograph the three bad brackets looked exactly like the good ones. Not similar. Identical, down to the last visible pixel.

The camera had been installed eight months earlier. In those eight months the maintenance team had replaced the diffuser on the light, and the new diffuser was a slightly different grade of acrylic. It scattered 12% less light. The automatic exposure on the camera had compensated, as it was designed to, by lengthening the exposure time from 4 milliseconds to 4.5. Everything stayed the same brightness. And the defect — a burr on an internal chamfer that was visible only as a shadow at a particular angle — stopped existing.

Nothing in that chain was a software failure. The software did exactly what it was told, on data that no longer contained the information it needed. This is the normal shape of a vision problem, and it is why this book starts with a sensor rather than a network.

Light Is a Count

A camera does not record brightness. It counts photons.

Inside the sensor, behind each lenslet and colour filter, there is a small well of silicon. A photon that lands in the well has a chance of knocking an electron loose, and the electron stays there until the camera reads it out. The number of electrons is proportional to the number of photons that arrived during the ex-

posure, and the number the camera reports is a digitised version of that count.

Two consequences follow immediately, and both matter more than any algorithm in this book.

The first is that brightness is a *rate*, not an amount. A dark scene exposed for a long time produces the same count as a bright scene exposed briefly. This is why a camera can photograph a black rubber seal and a white ceramic one and produce images of similar average brightness — and why that convenience is dangerous, because it erases the very difference that might have identified the material.

The second is that the count is finite. A typical industrial sensor well holds somewhere between ten thousand and fifty thousand electrons before it saturates. Once saturated, further light adds nothing, and the pixel reports its maximum value. Everything brighter than that limit becomes the same flat white. In a photograph of a shiny machined surface, the specular highlights are not bright; they are *uninformative*, and no amount of processing will recover texture that was clipped at the top of the well.

Quantity	Typical value	What it means in practice
Full well capacity	10,000 electrons	Brightest usable signal before clipping
Read noise	3 electrons	Floor of the measurement, per pixel
Dark current	1 electron per second at 25 °C	Grows with temperature and exposure
Bit depth	8 or 12 bits	256 or 4,096 reported levels

Counting Has a Square Root in It

Photon arrival is a random process. If a pixel receives an average of 10,000 photons, it will not receive exactly 10,000 every time; it will receive 10,000 give or take about 100, because the standard deviation of a count is its square root.

That rule, noise equals the square root of the signal, is the most useful single fact in practical vision. It says that bright pixels are proportionally cleaner than dark ones:

$$\text{relative noise} = 1 / \sqrt{(\text{number of electrons})}$$

At 10,000 electrons the relative noise is 1 part in 100, or 1%. At 400 electrons it is 1 part in 20, or 5%. At 100 electrons it is 10%.

Now put that beside a typical defect. A scratch on a brushed steel plate might reduce the local brightness by 8 grey levels out of 255. If the surface is well exposed and sitting at 180 grey levels, the pixel noise is around 4 levels, and the scratch is two noise-widths deep — detectable, marginally, with filtering. If the same surface is underexposed by two stops and sits at 45 levels, the noise is still around 4 levels, and the scratch is now a single noise-width: the signal has shrunk and the noise has not. The defect has not become smaller. It has become invisible.

This is the reason an engineer will spend an afternoon on a light and ten minutes on a threshold. Contrast is a physical quantity, bought with photons; thresholds only decide where to cut.

Twelve Bits, Eight Bits, and the Cost of Rounding

Industrial cameras usually digitise to 12 bits, giving 4,096 levels, and then hand you 8-bit images because every library expects them. The conversion is a division by sixteen, and it throws away 94% of the available levels.

Usually that is fine, because noise occupies several of those levels anyway: there is no point preserving distinctions finer than the measurement uncertainty. It becomes a problem in three situations.

Low-contrast work. If the entire part occupies 20 grey levels in the 12-bit image — a dark component on a dark background — dividing by sixteen compresses it into one or two levels, and the shape dissolves into quantisation steps. Keep the 12-bit image, or stretch it before converting.

Gradient work. When you compute a derivative across neighbouring pixels, you are subtracting numbers that are nearly equal. Rounding error, which was invisible in the image, becomes a large fraction of the difference.

Thresholds near a peak. Histograms with a narrow mode become comb-like after aggressive rounding, and a threshold that looked stable starts jumping.

The practical rule: convert late, and never convert twice. Do the arithmetic in the widest format the camera supplies, and quantise only when you must display or store.

Three Numbers Are Not a Colour

A colour camera puts a red, green or blue filter over each pixel in a repeating pattern, usually two greens for every red and blue. The camera then invents the missing two values at each pixel by interpolating from neighbours, a step called demosaicing.

Two things go wrong for machine vision.

The first is resolution. A 5-megapixel colour sensor resolves detail at roughly the level of a 1.25-megapixel monochrome sensor, because three of every four pixels are measuring a different colour rather than the same one. For measurement, monochrome is almost always the better choice.

The second is that the channel values depend on the illuminant. The same white plastic reads as slightly blue under daylight and slightly orange under an incandescent lamp, and the camera's automatic white balance will "correct" it — silently changing the numbers your threshold depends on. Turn automatic white balance off. If colour matters, calibrate it with a reference patch in the field of view, and treat the channels as ratios rather than as absolute values.

Colour is genuinely useful when the *material* is the discriminator: copper versus brass, a red seal versus a black one, a label versus a bare can. It is nearly useless for geometry, where monochrome contrast and a good light will beat it every time.

The Three Dials

Every camera gives three controls that change exposure, and a beginner treats them as interchangeable because they all make the picture brighter. They are not interchangeable, and the table is worth memorising.

Control	Changes brightness	Changes noise	Changes blur
Aperture (f-number)	Halving the area costs one stop	Little, if light is plentiful	Changes depth of field
Exposure time	Linear	Longer helps, until motion intrudes	Linear smear
Gain (ISO)	Linear	Amplifies signal and noise together	None

Aperture is the only control that also changes what is in focus, and in a fixed rig it is usually set once and left. Exposure time is the control that interacts with

motion: a conveyor at half a metre per second moves 2 mm in 4 milliseconds, and 2 mm at a resolution of 0.28 mm per pixel is seven pixels of smear. Gain is the control that looks free and is not, because it multiplies the noise as faithfully as it multiplies the signal.

The sequence that works is: open the aperture as far as depth of field allows, set the exposure time from the motion budget, and add gain only to reach the brightness that the exposure time forbids.

What the Numbers Mean for the Rest of the Book

Every chapter that follows inherits the limits set here. A filter cannot recover a clipped highlight. A threshold cannot separate two materials that produce the same electron count. A network trained on images with automatic exposure has learned the camera's compensating behaviour as if it were a property of the part.

So the first question about any vision task is not "which algorithm" but "how many photons carrying the difference will land on how many pixels". If that question has no good answer, the honest response is to change the light, the lens, or the part's presentation — and to do it before writing a line of code.

Exercises

1. A pixel receives 2,500 electrons on average. What is the relative shot noise, in percent? What does it become if the exposure is halved?

2. A defect changes local brightness by 6 grey levels. The region sits at 60 grey levels out of 255. Estimate the pixel noise from the square-root rule (assume the sensor maps 10,000 electrons to 255) and say whether the defect is detectable without filtering.

3. A camera is set to automatic exposure and automatic white balance, and runs unattended for a year. List three ways its output could change without any part changing.

4. Explain, in two sentences, why increasing gain does not improve a marginal image, using the electron counts from this chapter.

5. A shiny aluminium part produces large white regions with no texture. Describe two physical changes that would give the camera something to

measure there.

6. You are asked to tell brass from copper on a conveyor. Would you use colour or monochrome with a coloured light? Give one advantage of each and pick one.

CHAPTER TWO

Pixels, Coordinates and the Cost of Sampling

A hairline crack, 0.06 millimetres wide, running across a cast housing. Under a microscope it is obvious. On the inspection image it is not there at all.

The camera was a 5-megapixel unit with a 12-millimetre lens at a working distance of 600 millimetres, and the housing filled the frame. The engineer measured the field of view with a rule: 660 millimetres across. He divided by the number of pixels across the sensor, 2,592, and wrote the answer on the whiteboard: 0.25 millimetres per pixel.

A crack two-tenths of a pixel wide does not make a dark line in an image. It makes each affected pixel very slightly darker than its neighbours — by an amount that depends on where the crack happens to fall inside the pixel's area. Move the part by a tenth of a millimetre and the effect changes. The crack is not hidden by the algorithm; it was never sampled.

This chapter is about that arithmetic, because it is the single calculation that decides whether a vision project is possible.

Resolution Is a Physical Quantity

Camera datasheets quote megapixels, which is a number about the sensor and says nothing about the task. The number that matters is the **spatial sampling interval**: millimetres of the real world per pixel, at the working distance you intend to use.

```
mm per pixel = (field of view width in mm) / (pixels across the sensor)
```

Three levers change it, and they are not equivalent in cost.

Move closer. Field of view shrinks in proportion to distance, so halving the distance halves the sampling interval and quadruples the effective resolution per unit area. This is the cheapest lever and the one most often blocked by the machine.

Use a longer lens. The same effect as moving closer, achieved without moving the camera: a lens of twice the focal length halves the field of view. It costs money, reduces depth of field, and requires a rigid mount.

Buy more pixels. Doubling the linear pixel count halves the interval without changing the optics, but it also halves the light per pixel and increases readout time, so it is rarely free in practice.

Configuration	Field of view at 600 mm	Pixels across	mm per pixel
12 mm lens, 1/1.8 in sensor	360 mm	1,280	0.28
50 mm lens, 1.1 in sensor	158 mm	4,112	0.039
12 mm lens, 1.1 in sensor	660 mm	2,592	0.25

The middle row is what the crack needed; the bottom row is what the project had.

How Many Pixels Does a Feature Need?

The sampling theorem says you need more than two samples per cycle to reconstruct a pattern, and that is the floor for signals designed to be reconstructed. Vision is harder than that, because a defect has to be *detected* against noise, not merely represented.

The working rules used in practice:

- Three pixels across the smallest feature you must detect, as an absolute minimum.
- Five pixels across it for reliable measurement of its size.
- Ten pixels across it if you intend to measure position to a tenth of the feature's width.

These are not laws but they are remarkably durable, and they are cheap to apply. A 0.2 mm burr at 0.28 mm per pixel is 0.7 pixels — hopeless. The same burr at 0.04 mm per pixel is five pixels — comfortable. The project's fate was decided by a division, before anyone chose a network.

Aliasing: The Failure That Looks Like a Different Problem

When a pattern finer than twice the sampling interval is imaged, it does not disappear. It reappears as a coarser pattern that is not in the object at all. This is aliasing, and in industrial images it takes three familiar forms.

Moiré on woven or perforated material. A mesh screen with 0.4 mm openings sampled at 0.28 mm per pixel produces broad wavy bands. An inspector looking at the image sees a defect that is an artefact of sampling; a real defect may be invisible underneath it.

Broken edges on fine pitches. A connector with 0.3 mm contacts sampled at 0.25 mm per pixel produces contacts of wildly different apparent widths, some darker, some lighter, none corresponding to the physical part.

Crawling patterns in video. The same mesh, viewed live, shimmers as the part moves — a signature that is almost diagnostic of undersampling.

The cure is optical or mechanical, not digital: fewer pixels per feature means blur first. Slightly defocusing the lens, or fitting a diffuser, removes the high frequencies before they are sampled, and a blurred honest image is far better than a sharp aliased one. Software anti-aliasing after sampling cannot undo the damage, because the information is already gone.

Coordinates, and the Half-Pixel That Costs a Shift

Every image library puts the origin in a different place, and every one of them has been the cause of a rejected batch.

The convention that makes the arithmetic consistent: a pixel is an *area*, not a point. Pixel (0, 0) covers the square from 0.0 to 1.0 in both axes, and its centre is at (0.5, 0.5). A blob whose centroid computes to (100.0, 50.0) is therefore at the corner shared by four pixels, and a blob at (100.5, 50.5) is at the centre of one.

Why it matters, in millimetres: at 0.28 mm per pixel, a half-pixel disagreement between two pieces of code is a 0.14 mm disagreement in reported position. If a robot is asked to place a part with a ±0.1 mm tolerance, the convention alone can consume the budget.

Three habits prevent the class of error:

Write the convention down once, in the project notes, and refer to it. "Centroid coordinates are pixel centres; origin at the centre of the top-left

pixel" is a sentence worth its ink.

Test with a synthetic image. Draw a disc whose centre you know exactly, run the centroid routine, and check it returns that centre to within 0.01 pixels. This takes twenty minutes and catches the error before it reaches a machine.

Never mix libraries silently. If one routine returns corners and another returns centres, convert at the boundary, in one function, with a comment.

Resizing Is Not Free

Downsampling an image to speed up processing is one of the most common optimisations, and one of the most common ways to delete the defect.

A 1,280 by 1,024 image reduced to 320 by 256 has one output pixel for every sixteen input pixels. Each output is some average of its block. A one-pixel-wide scratch covering 2% of a block contributes 2% of the block's brightness change: in an 8-bit image where the scratch was 40 levels darker than its surroundings, the downsampled difference is 0.8 levels, which is below the rounding step.

Upsampling has the opposite problem: it invents smooth values between measurements and makes the image look more precise than it is. Interpolation is a display convenience, not an increase in information.

Operation	Cost	Information
Nearest-neighbour downscale	Cheapest	Discards, badly
Area-average downscale	Cheap	Discards, fairly
Bilinear upscale	Cheap	Invents, smoothly
No resize, larger sensor	Expensive	Gains

If the pipeline needs to be faster, crop before you shrink. A region of interest of 400 by 300 pixels contains 120,000 pixels instead of 1.31 million — eleven times less work, and every remaining pixel is a real measurement.

Bandwidth Is Part of the Design

Numbers make the constraint concrete. A 1,280 by 1,024 8-bit image is 1.31 megabytes. At thirty frames per second that is 39 megabytes per second, which is comfortable for a Gigabit Ethernet link and awkward for USB 2. A 12-bit image is 1.97 megabytes, or 59 megabytes per second, which requires a faster link or a smaller frame.

Compression is a temptation. Lossy compression such as JPEG achieves ten-to-one reductions by discarding high-frequency detail — exactly the detail that a scratch or an edge consists of, and with visible blocking artefacts near high-contrast boundaries. For a photograph shown to a human it is excellent. For a measurement, use a lossless format or no compression at all, and if storage is the constraint, store fewer frames of better quality.

Things to Try

Take a ruler, a camera and a textured surface — a coin, a printed page, a piece of woven cloth. Photograph it at your normal working distance with an object of known size in the frame. Compute millimetres per pixel yourself, from the image, not from the datasheet. Then measure something small in the image and compare with a caliper. The gap between the two numbers is the beginning of metrology.

Next, photograph the same cloth from far enough away that the weave is finer than a pixel, and look for moiré. Then defocus slightly and photograph it again. Compare. That pair of images explains more about sampling than any chapter can.

Finally, list three features in your own work or study that must be detected, and compute the sampling interval each one needs at three, five and ten pixels across. Compare against the camera you have. The answer is sometimes "this needs a different camera", and it is better to know that in week one.

Problems

1. A sensor has 2,448 by 2,048 pixels and a 8.4 mm by 7.0 mm active area. With a 25 mm lens at 400 mm working distance, what is the field of view, and what is the sampling interval?

2. A defect is 0.15 mm wide. What sampling interval do you need for five pixels across it? What field of view does that imply for a 2,448-pixel sensor?

3. An image is downsampled from 2,048 by 2,048 to 512 by 512, then a 3 by 3 edge filter is applied. Estimate the smallest feature the filter can still respond to, in original pixels.

4. You are told the position of a part must be reported to ±0.05 mm, and the sampling interval is 0.2 mm. Is sub-pixel estimation needed? What accuracy would you expect from a simple centroid?

5. A conveyor runs at 0.8 m/s and the camera exposes for 3 ms. How far does the part move during the exposure, in millimetres and in pixels at 0.2 mm per pixel? What exposure time would keep the smear under one pixel?

6. Explain to a colleague, in plain language and without formulas, why blurring an image can make a fine pattern easier to interpret.

CHAPTER THREE

Convolution: The One Operation Everything Uses

Blur a photograph. Sharpen a photograph. Find its edges. Remove its noise. Detect a texture, measure a gradient, feed a convolutional network. Five different verbs, one arithmetic: a small grid of numbers slid across the image, multiplying and adding as it goes.

It is worth learning properly, because every vision system you will meet is either doing this or describing itself in its vocabulary. The operation is also the last piece of mathematics in this book that can be done entirely on paper in under a minute, so it is the last place where you can check your understanding by hand.

The Kernel

A **kernel** — also called a filter, a mask, or a window — is a small grid of numbers, usually 3 by 3 or 5 by 5. The **convolution** places the kernel over a neighbourhood of the image, multiplies each image value by the kernel value underneath it, adds the products, and writes the total as one output value.

Take this patch of a grayscale image, with a single bright pixel in the middle:

```
30   30   30
30  200   30
30   30   30
```

And this kernel, the average of a 3 by 3 neighbourhood, in which every weight is one ninth:

kernel = (1/9) x [[1,1,1],[1,1,1],[1,1,1]]

The output for the centre position is

(30 + 30 + 30 + 30 + 200 + 30 + 30 + 30 + 30) / 9 = 440 / 9 = 48.9

The bright pixel, which was 200, becomes 49. Its eight neighbours, which were 30, become 49 as well. The spot has not been removed; it has been spread over nine pixels, and the peak has fallen by three quarters.

That is blurring, and it is worth noticing that it is *lossy by design*. The kernel cannot know that the bright pixel was a defect and not noise. It treats every neighbourhood alike.

A Kernel That Behaves Differently

Averaging treats the centre and the corners equally. A Gaussian kernel does not: it weights the centre most and the corners least, on the reasonable assumption that nearby pixels are more related than distant ones. The standard 3 by 3 approximation is

```
1   2   1
2   4   2
1   2   1
```

with a divisor of 16, the sum of the weights. On the same patch:

(30 + 60 + 30 + 60 + 800 + 60 + 30 + 60 + 30) / 16 = 1,160 / 16 = 72.5

Compare the two answers: mean gives 48.9, Gaussian gives 72.5. The Gaussian has smoothed less, keeping more of the peak, because it trusted the centre value more. That is the whole trade in one pair of numbers. A Gaussian blurs more gently and preserves structure slightly better; a box filter is cheaper and has a characteristic square artefact.

The divisor is not decoration. A kernel whose weights do not sum to the divisor will brighten or darken the image, and a kernel whose weights sum to zero will produce an image of edges only — which is exactly what the next chapter does.

Borders, and the Dark Frame

A 3 by 3 kernel cannot be centred on a pixel at the very edge of the image, because two of its neighbours do not exist. Every library solves this with **padding**: it invents values outside the image.

Zero padding is the simplest and the worst. The invented neighbours are black, so the border pixels are dragged towards zero, and the result is a dark

one-pixel frame around every filtered image. On a large image nobody notices. On a small region of interest, or when you filter repeatedly, the frame contaminates the measurement.

Other options, in increasing order of quality: replicate the edge value outwards; reflect the image at its border; wrap it around. Replication is the usual default in industrial libraries because it neither darkens nor invents a symmetry that is not there.

The general rule: filter inside a region, then measure in the interior. If you need values at the boundary, crop the region of interest slightly larger than you need and discard the filtered margin.

Sharpening Is a Subtraction

If blurring is a kernel, sharpening is blurring used against itself. The classic **unsharp mask**:

```
sharpened = original + amount x (original - blurred)
```

The bracket is the detail that the blur removed. Adding it back exaggerates local contrast, which the eye reads as sharpness.

Work it through on the bright pixel. With amount = 1 and the Gaussian above: 200 + (200 - 72.5) = 327.5. That value does not exist in an 8-bit image, and it clips to 255. The pixel saturates, the surrounding ring is darkened by the subtraction, and the result is a hard white dot with a black halo.

That halo is not a bug in your code. It is what sharpening does when pushed past the available range, and it is why sharpening is a display tool rather than a measurement tool. If you sharpen before measuring an edge position, you have moved the edge by an amount that depends on the amount setting.

Separability: Why Blurring Is Cheaper Than It Looks

A 3 by 3 kernel needs nine multiplies per pixel. A 7 by 7 kernel needs forty-nine. That grows quickly, and it is why real systems exploit a property of the Gaussian: it can be written as two one-dimensional passes.

```
[[1,2,1],[2,4,2],[1,2,1]] = [[1],[2],[1]] x [[1,2,1]]
```

So a 7 by 7 Gaussian can be applied as a 7-tap horizontal pass followed by a 7-tap vertical pass: fourteen multiplies instead of forty-nine, a saving of 3.5 times.

Kernel size	Direct cost per pixel	Separable cost	Saving
3 by 3	9	6	1.5x
5 by 5	25	10	2.5x
7 by 7	49	14	3.5x
15 by 15	225	30	7.5x

The saving is the reason large blurs are affordable at frame rate, and the property is worth checking before you implement any kernel by hand: if it can be factored into a row and a column, factor it.

What a Filter Costs at Frame Rate

A 1,920 by 1,080 image has 2.07 million pixels. A 3 by 3 filter at 9 multiply-adds per pixel is 18.7 million operations per frame; at thirty frames per second that is 560 million per second. A modern processor does that without noticing, which is why filtering seems free.

It stops being free when the kernels multiply. A convolutional layer with thirty-two 3 by 3 filters over a 640 by 640 three-channel image performs 640 x 640 x 32 x 27 = 354 million multiply-accumulates for one frame. That number, not the elegance of the architecture, is what decides whether your system runs on a CPU or needs an accelerator.

The arithmetic is also where the practical wisdom lives: crop before you filter, build pipelines that stop early when a cheap test answers the question, and do not run a 15 by 15 blur on a 12-megapixel image if a 5 by 5 on a quarter-sized region will serve.

Two Things Convolution Cannot Do

It cannot remove an impulse of unbounded size. A median filter can, because it takes the middle value of the neighbourhood: on the patch above, the median of the nine values is 30, so the bright pixel is deleted entirely rather than spread. Median filtering is nonlinear, it is not a convolution, and for saltand-pepper noise it is the correct tool.

It cannot preserve an edge and smooth a region with the same kernel. This is the oldest tension in the subject; the bilateral filter and edge-preserving variants exist precisely to weaken the trade.

Working Notes

1. Compute the mean-filtered and Gaussian-filtered values at the centre of this patch, with the kernels given above: 10, 10, 10, 10, 250, 10, 10, 10, 10. Which filter would you use to find a small bright defect, and why?

2. A kernel is [[0,1,0],[1,-4,1],[0,1,0]]. Its weights sum to zero. What will the output look like on a flat grey region? On an edge?

3. A colleague applies a 5 by 5 mean filter five times to clean up an image. What has happened to a 2-pixel-wide dark line after the third pass?

4. A 7 by 7 Gaussian must run on 4,096 by 3,000 images at 10 frames per second. Compute the operations per second for direct and separable implementations.

5. Padding with zeros produces a dark border. Describe what a threshold applied to the filtered image would do at the border, and how you would prevent it.

6. Write down a 3 by 3 kernel that would detect a horizontal line one pixel thick and nothing else. Test it on paper against a vertical line.

PART TWO

Structure Without Learning

CHAPTER FOUR

Edges, Gradients and the Honest Line

The task was to measure the width of a weld seam to a tenth of a millimetre, on a part that had already been welded. The camera was adequate, the light was a bar above the fixture, and the software was a commercial package with an edge tool that let the operator drag a box across the seam and read a width.

It worked in the afternoon and failed in the morning, and the difference between afternoon and morning was a window.

By three o'clock the sun came round to the west side of the building and added a soft gradient of light across the fixture. The seam's left edge darkened; the right edge brightened. The edge tool found both edges, exactly as designed, and reported a width that was 0.4 millimetres too large. The tool was not confused. It was an honest detector reporting a dishonest signal.

Edges are the first structure most people extract from an image, and the most misunderstood. An edge is not a property of an object. It is a property of the light that happened to fall on the object when the shutter opened.

What an Edge Is

An edge is a place where brightness changes quickly. In a continuous world, that change is a slope; in a digital image, it is a difference between neighbouring pixels.

The simplest estimate of the horizontal slope at a pixel is the difference between its right and left neighbours:

```
Ix = I(x+1) - I(x-1)
```

This "central difference" is cheap and surprisingly good, because it averages over two pixels and so cancels some noise. Its vertical twin gives Iy. Together they form the **gradient**, a vector with two components.

A 3 by 3 version weights the near neighbours more heavily than the diagonal ones, and this weighted version is the Sobel operator:

```
-1    0   +1
-2    0   +2
-1    0   +1
```

A Worked Response

Take the patch from the previous chapter, this time with a clean vertical step: dark on the left, bright on the right.

```
30   30  120
30   30  120
30   30  120
```

Apply the horizontal Sobel kernel to the centre pixel:

```
(-1)(30) + (0)(30) + (+1)(120) =   90
(-2)(30) + (0)(30) + (+2)(120) =  180
(-1)(30) + (0)(30) + (+1)(120) =   90
total Gx = 360
```

Now the vertical kernel, which is the horizontal one turned on its side:

```
-1   -2   -1
 0    0    0
+1   +2   +1

(-1)(30) + (-2)(30) + (-1)(120) = -210
( 0)(30) + ( 0)(30) + ( 0)(120) =    0
(+1)(30) + (+2)(30) + (+1)(120) =  210
total Gy = 0
```

The gradient magnitude is the hypotenuse: $\sqrt{(360^2 + 0^2)} = 360$. Its direction is horizontal, pointing right — across the edge, which is always the case, because that is the direction in which brightness changes fastest.

The raw number depends on the kernel's weights, so it is customary to divide by the sum of the positive weights to bring it back into image units. For Sobel that sum is 4 in each column, giving 360/8 = 45: the step was 90 grey levels, and the normalised response reports half of it. The factor of two is an artefact of the weighting, not a physical law, which is why thresholds should be set on your own images and never copied from a textbook.

From Gradient to Edge Map

The gradient magnitude field is an image in its own right: bright where structure exists, dark where the picture is flat. Turning it into a set of lines requires two decisions.

Threshold. Above the threshold, call it an edge; below, discard. Choose the threshold by looking at a histogram of gradient magnitudes on a real image, and pick a value in the valley between the noise and the structure. If there is no valley, the image does not support the task.

Thinning. A step edge produces a ridge several pixels wide, because both the rising and falling filter responses exceed the threshold. **Non-maximum suppression** keeps only the local maximum along the gradient direction: for each pixel, compare its magnitude with the two neighbours on either side across the edge, and delete it if it is not the largest.

The Canny detector combines these with one more idea: **hysteresis**. Two thresholds are used, a high one that starts an edge and a low one that continues it. A strong edge is kept only if it connects to another strong edge. The effect is to suppress isolated noise responses while keeping faint continuations of genuine boundaries — and the effect is dramatic on textured surfaces, where a single threshold either drowns in texture or loses the object.

Step	Question answered	Typical failure if skipped
Smooth	Is the pixel noise below the step size?	Thousands of spurious edges
Gradient	Where does brightness change?	Nothing found
Magnitude threshold	Which changes are real?	Noise retained or structure lost
Non-maximum suppression	Where exactly is the edge?	Two-pixel-wide bands
Hysteresis	Is this edge connected to a real one?	Isolated specks

Where Is the Edge, Exactly?

For measurement, the interesting question is not whether an edge exists but where its centre lies, to a fraction of a pixel.

A blurred step has a transition width of roughly four times the blur radius. Fit a curve to the gradient profile — a straight line through the two steepest points is usually enough — and the crossing of its midpoint gives a position accurate to about a tenth of a pixel on a clean, well-lit, straight edge.

That "clean, well-lit, straight" is doing heavy work. On a curved edge the fit must account for curvature; on a low-contrast edge the noise dominates; and on a surface with visible microstructure the profile is not a step at all but a stutter.

The measurement to report alongside the width is therefore not just a number but a **repeatability**: take fifty images of the same part without moving it, compute the width each time, and report the standard deviation. If that number is 0.15 mm, then a specification of ±0.1 mm is unmeetable and the argument is over before it starts. This one habit — measuring the measurement — prevents more bad projects than any algorithm choice.

The Four Ways Edges Lie

Lighting gradients. A smooth change in illumination across the field produces a gradient that is not on the object at all. It shifts every edge it touches in the same direction, so widths stay roughly right while positions drift. Repair: diffuse the light, or calibrate it with a blank reference image and divide it out.

Glare and specular reflection. A shiny surface reflects the light source rather than the surface. Edges vanish inside the reflection, and new edges appear at its boundary. Repair: change the geometry — use a dome light or a polariser — rather than the threshold.

Texture. A brushed or cast surface has thousands of real micro-edges. A low threshold finds them all; a high one loses the object. Repair: blur along the edge direction before differentiating, which averages the texture while preserving the boundary.

Curvature and perspective. An edge that is straight in three dimensions may be curved by the lens, and a width measured in pixels is not a width in millimetres unless the scale is known at that location. Repair: calibrate, and measure in millimetres rather than pixels.

None of these is fixed by a better detector. All four are fixed by the rig, which is why Part Four exists.

Problems

1. Compute Gx and Gy for the centre of this patch with the Sobel kernels: rows 40, 40, 160 / 40, 40, 160 / 40, 40, 160. What is the magnitude, and in which direction does the gradient point?

2. The same patch is imaged again with a gain of 1.5 applied. What happens to the gradient magnitude, and what does that tell you about using a fixed threshold across two cameras?

3. Explain why non-maximum suppression compares along the gradient direction rather than along the row and column.

4. A blurred edge has a transition 6 pixels wide. Estimate the blur radius, and the smallest feature width that will still produce a distinct pair of edges.

5. An edge tool reports a width of 12.4 mm with a repeatability of 0.08 mm over fifty static images. The tolerance is ±0.1 mm. Is the tool adequate? What would you check before answering?

6. Design a lighting change that would remove the gradient in the opening story without moving the camera or the part.

CHAPTER FIVE

Corners, Features, and the Points That Survive

Two photographs of the same printed circuit board, taken a second apart from a camera that has not moved. Between them the board has been nudged by a handling robot, rotated by perhaps two degrees, and lit slightly differently because a shadow moved.

A template matcher compares the two and fails. The template is a rectangle of pixels; rotating the board rotates the pixels inside it; the comparison is now between two arrangements that no longer line up. Everything correlates less, the peak score falls, and the system reports no match.

A feature matcher compares the same two and succeeds, because it does not compare pixels. It finds a few hundred distinctive points, describes each by its local neighbourhood in a way that is stable under rotation and moderate changes of brightness, and then finds the arrangement of descriptors that best explains both images. From that arrangement it computes the rotation, the translation, and the scale.

This is the idea that carries machine vision from "measuring a part that always appears in the same place" to "understanding a scene". It is also the idea that underlies modern three-dimensional reconstruction, panorama stitching, and the alignment step inside many industrial systems.

Three Kinds of Neighbourhood

Look at a small window of an image and ask how it would change if you slid it by one pixel in each direction.

Flat region. Nothing changes. The window has no information about where it is, and no amount of matching will localise it.

Edge. Sliding along the edge changes nothing; sliding across it changes a lot. The window is localised in one direction only — the classic aperture problem, where a straight edge tells you it has moved but not how far along itself.

Corner or texture. Sliding in any direction changes something. The window is localised in both directions, and that is what makes it usable as a landmark.

The Harris corner detector turns this description into arithmetic. For each pixel, sum three quantities over a small window: the squared horizontal gradient, the squared vertical gradient, and their product.

$$Sxx = \text{sum of } Ix^2,\ Syy = \text{sum of } Iy^2,\ Sxy = \text{sum of } Ix.Iy$$

Then form a 2 by 2 matrix from those three numbers and compute two values that summarise its shape:

$$R = Sxx.Syy - Sxy^2 - k.(Sxx + Syy)^2$$

with k around 0.04. Large positive R means a corner. Negative R means an edge. Small values mean flat.

Worked, on a hypothetical window: suppose Sxx = 100, Syy = 100 and Sxy = 0. Then

$$R = 10{,}000 - 0 - 0.04 \times (200)^2 = 10{,}000 - 1{,}600 = 8{,}400$$

A corner, strongly. Now suppose the window lies on a horizontal edge: Sxx = 100, Syy = 0, Sxy = 0.

$$R = 0 - 0 - 0.04 \times (100)^2 = -400$$

Negative: an edge, to be rejected.

The two numbers are not measurements of anything physical. They are a compact way of asking the sliding-window question, and they are worth computing once by hand on a small patch so that the response map stops being a black box.

Descriptors: Making a Corner Recognisable

A corner is a location. To match corners between images, each needs a description that is stable when the image changes in the ways images change: rotation, scale, brightness, small perspective shifts, noise.

The standard recipe has three parts.

Orientation. Compute the dominant gradient direction in the neighbourhood, using a histogram of gradient angles weighted by magnitude. Rotate the description to that direction, so that the same physical corner produces the same descriptor regardless of how the camera was rotated.

Sampling. Divide a square region around the point into a grid of sub-regions — typically 4 by 4 — and record, for each, a summary of the gradient directions within it. The result is a vector of numbers, usually 128 of them, which is the descriptor.

Normalisation. Scale the vector so its length is one, and clamp the largest components. This makes the descriptor insensitive to overall brightness and contrast changes, which is exactly what a change of lighting produces.

Then matching is a matter of distance. For each descriptor in the first image, find the nearest and second-nearest in the second image, and accept the match only if the nearest is clearly closer. The **ratio test** is the standard guard:

Nearest distance	Second nearest	Ratio	Decision
40	60	0.67	Accept
40	45	0.89	Reject
25	90	0.28	Accept, confidently

A threshold of 0.8 is conventional. The logic is worth stating: a descriptor that is equally close to two different features is probably ambiguous, and a wrong match is more expensive than a missing one.

Turning Matches into a Transformation

Matches alone are a cloud of correspondences, some of them wrong. The next step fits a geometric model and rejects the matches that do not obey it.

For a flat object viewed from a fixed camera — a board on a table — the appropriate model is a **homography**, an eight-parameter mapping that accounts for rotation, translation, scale, shear and perspective. Four point correspondences determine it exactly.

Because some matches are wrong, the fitting is done by consensus rather than by least squares. **RANSAC** picks four matches at random, computes the transformation they imply, and counts how many of the remaining matches agree with it within a tolerance. It repeats this many times and keeps the trans-

formation with the most agreement.

The arithmetic explains why the repetition is necessary. Suppose 60% of the matches are correct. The chance that a random set of four is entirely correct is 0.6^4 = 0.13, so a single attempt usually fails. After ten attempts the chance of at least one clean set is 1 - (1 - 0.13)^10 ~ 0.75. After forty attempts it is 1 - 0.87^40 ~ 0.996. Forty attempts cost almost nothing, which is why RANSAC is set to hundreds rather than tens in practice.

Inlier fraction	P(clean sample of 4)	Attempts for 99% success
0.3	0.008	570
0.5	0.063	71
0.6	0.130	34
0.8	0.410	9

The table is a useful diagnostic in reverse: if a system needs a thousand attempts to find a consistent transformation, the matches are mostly wrong, and the place to look is the descriptor quality or the lighting, not the attempt count.

Where Features Fail

Feature methods are robust to more than template matching and less than intuition suggests. Four failures recur.

Repetitive texture. A grid of identical holes, a woven fabric, a brick wall. Every descriptor looks like every other, the ratio test rejects almost everything, and the survivors are arbitrary. Repair: add a non-repetitive marker, or use a method that exploits structure rather than points.

Scale change beyond the descriptor's range. A printed label photographed at twice the distance has features twice as large. Descriptors handle this by being computed at several scales, but the range is finite, and a factor of four exceeds it. Repair: constrain the working distance, or use a scale-invariant pipeline explicitly.

Motion blur and defocus. Blur destroys the fine gradients that descriptors depend on. A feature map from a blurred image has few points and poor localisation. Repair: shorten the exposure, or accept fewer points and rely on geometry.

Reflective and transparent surfaces. A specular highlight moves with the light, not with the object, so the same physical point produces different descriptors in successive frames. Repair: polarisation, diffuse lighting, or features chosen away from highlights.

Note the shape of these repairs. Not one of them is "use a better matcher". They are all changes to the physical situation or to the assumptions, which is the recurring theme of this book.

What Features Are Good For

The classical uses remain valuable and are often cheaper than a network.

Alignment before measurement. Locate the part by its features, then measure in a coordinate frame attached to the part rather than to the camera. This single trick removes most of the fixture's tolerance from the measurement budget.

Stitching. Overlapping images of a large surface, aligned by shared features, produce a mosaic wider than any single field of view.

Three-dimensional reconstruction. Correspondences across two views, plus calibration, give depth by triangulation — the subject of Chapter Seven.

Verification of change. Features present in a reference image and absent in a test image, in the same geometric arrangement, are evidence of a missing component.

In each case, the value lies in the *arrangement* rather than in any single point. A hundred points that all agree on a rigid transformation are far stronger evidence than any one of them, and this is a statistical argument that survives long after the specific detector has been superseded.

Things to Try

Photograph a page of text from directly above, then from an angle of about thirty degrees. Run a corner detector on both — most image libraries have one built in — and compare the number and distribution of points. Then rectify the angled image with a homography and check whether the text becomes readable. That sequence, in fifteen minutes, is the whole classical pipeline.

Next, photograph a tiled floor or a brick wall and run the same detector. Look at how the points distribute: they will cluster at the same relative position

in every tile, which is exactly the repetition problem described above. Understanding why the matcher fails there is more instructive than any amount of reading about robustness.

CHAPTER SIX

Segmentation: Regions, Thresholds and the Shape of Things

The parts arrived on a tray, nine of them, black plastic with a small locating hole in each. The counting system was a straightforward piece of work: threshold the image, find connected regions, count the ones above a size limit. It had been in service for a year and had never been wrong.

Then the mould was reconditioned, and the parts came out slightly shinier. On three of the nine, a specular highlight sat exactly where the moulding tool left a smooth patch. The highlight was bright enough to exceed the threshold. Each of those three parts now produced two regions instead of one, and the count read twelve.

The threshold was not wrong. The lighting was not wrong. The *definition of a region* — the assumption that a part is one connected blob of one brightness class — had stopped being true.

Thresholding, and Where the Number Comes From

A global threshold turns a grayscale image into a binary one: pixels above the value become 1, pixels below become 0. It is the simplest segmentation there is and, used well, one of the most reliable.

Where does the number come from? Three sources, in descending order of quality.

A physical argument. If the illumination is controlled and the reflectances are known, the threshold corresponds to a reflectance. Under a backlight, a hole in a metal sheet transmits nearly all the light and the metal nearly none, so any threshold in the middle of the range separates them, and the exact value does not matter. This is the situation to aim for, because it is the only one that stays correct when the parts change.

A histogram valley. Plot the distribution of pixel values. A good image for thresholding is bimodal: two humps with a valley between. Put the threshold at

the bottom of the valley, as far from both modes as possible.

A formula. Otsu's method chooses the threshold that maximises the variance *between* the two groups it creates. It is worth doing once by hand, because it makes visible what the algorithm assumes.

Otsu by Hand

Ten pixels, values 30, 35, 40, 45, 50, 160, 170, 180, 190, 200. The overall mean is 110.

For each candidate threshold, split the pixels into those at or below it and those above, compute each group's mean and weight, and evaluate

$$\text{between-class variance} = w_0 \times w_1 \times (m_0 - m_1)^2$$

Candidate	w0	m0	m1	Between-class variance
30	0.10	30.0	118.9	711
45	0.40	37.5	158.3	3,504
50	0.50	40.0	180.0	4,900
160	0.60	60.0	185.0	3,750
180	0.80	88.8	195.0	1,806

The maximum is 4,900, at a threshold of 50 — and, note, at every threshold between 50 and 100, because the data has a gap there and no pixels fall inside it. The method is not confused; the answer is genuinely non-unique, and any value in the gap separates the two groups perfectly.

That plateau is more informative than a single number. When Otsu returns a threshold in the middle of a wide empty gap, the image is easy and the task is robust. When the variance curve has a shallow peak and no gap, the image is marginal, and you should expect the result to depend on the day.

What Thresholding Cannot Do

Two classes only. Three materials of different brightness need two thresholds, or a different method.

No spatial information. Each pixel is decided alone, so noise becomes speckle and a smudge becomes an island.

No notion of an object. A part with a hole yields two regions, and a part with a highlight yields three.

The repairs for all three are the same family: use neighbourhood information. Morphology cleans the shape; connected components assembles the shape; local thresholds handle uneven lighting.

Morphology, on a Small Grid

Two operations, defined on a binary image and a small structuring element (usually 3 by 3).

Erosion keeps a pixel only if the entire structuring element fits inside the foreground. It shrinks objects and deletes anything thinner than the element.

Dilation sets a pixel if any part of the element touches foreground. It grows objects and closes small gaps.

Take this 5 by 5 patch, with a 3 by 3 block of ones surrounded by zeros:

```
0  0  0  0  0
0  1  1  1  0
0  1  1  1  0
0  1  1  1  0
0  0  0  0  0
```

Eroding with a 3 by 3 square of ones leaves exactly one pixel: the centre. Every other foreground pixel has at least one zero in its neighbourhood.

Dilating the original fills the whole 5 by 5 patch: every pixel is within one step of the block.

Chaining them gives the two workhorses. **Opening** (erode, then dilate) removes small protrusions and isolated specks while leaving large shapes roughly the same size. **Closing** (dilate, then erode) fills pinholes and joins pieces separated by a thin gap — which is exactly the repair for the specular highlight, provided the gap is smaller than the element.

The size of the element is a physical statement: "anything smaller than 0.9 mm is noise". Express it in millimetres, and check it against the real feature sizes, or it will eventually delete a real defect.

Connected Components

Once the image is binary, **connected components** labels each group of touching foreground pixels. Two flavours: 4-connectivity (up, down, left, right) and 8-connectivity (including diagonals). The choice changes the answer on diagonal structures — a thin diagonal line is one component under 8-connectivity and many under 4.

From each labelled region you can compute area (in pixels, then in square millimetres), centroid, bounding box, orientation, and a hundred shape descriptors. Two cautions.

Area in pixels is not area in millimetres unless the scale is uniform. With perspective it is not, and with a tilted part it is definitely not.

Perimeter and area are biased by the threshold. Raising the threshold shrinks every region slightly, and the bias is not uniform: a thin feature loses proportionally more than a thick one. Report the sensitivity: how much does the measured area change when the threshold moves by ten levels? If the answer is 8%, the measurement is a property of your threshold, not of the part.

Colour, and the Space You Choose

Segmenting by colour is often the fastest route when the discriminator is hue: a red seal, a blue label, a copper contact.

Do it in a space where hue is separated from brightness, such as HSV, rather than in RGB, where a change in illumination moves all three channels together. In HSV, the hue channel wraps at 360 degrees, which means a red object spans values near 0 and near 180 — a detail that produces the classic bug of half an object being segmented and the other half not.

For materials rather than pigments, prefer ratios: the red-to-green ratio of copper is stable across lighting levels in a way that raw red is not. And if the lighting can be controlled, a coloured light is often better than a colour camera: illuminate with red, and a red part becomes bright while everything else stays dark, with no need for colour processing at all.

When Objects Touch

Watershed segmentation treats brightness as a landscape and floods it from the minima. It is the standard answer to touching objects — overlapping nuts in a bin, adjacent cells under a microscope — and it depends entirely on finding good markers.

The practical version: threshold roughly to get the foreground, compute a distance transform (each foreground pixel's distance to the nearest background), and use the local maxima of that transform as seeds. The transform turns a dumbbell shape into two peaks joined by a saddle, and the watershed splits it there. If the peaks merge — genuinely touching objects of similar size — the method will not separate them, and the honest fix is to change the presentation so they do not touch.

Exercises

1. Compute the between-class variance for these six values at thresholds 40 and 100: 20, 25, 30, 150, 160, 170. Which is the Otsu threshold?

2. A blob of 4,800 pixels at 0.05 mm per pixel: what is its area in square millimetres? If the threshold is raised so the blob shrinks by one pixel on every side, what is the new area, and what is the percentage error?

3. Erode and then dilate this 5 by 5 patch by hand, using a 3 by 3 square: rows 0,0,0,0,0 / 0,1,0,0,0 / 0,1,1,1,0 / 0,0,1,0,0 / 0,0,0,0,0. What survives?

4. Explain why closing fixes the specular highlight in the opening story, and name the condition under which it would not.

5. A line-scan system images a 300 mm wide strip at 0.1 mm per pixel. A defect must be detected if its area exceeds 0.5 mm². How many pixels is that, and what morphological opening size is safe?

6. Design a lighting setup for counting black plastic parts on a tray, such that the parts cannot break into two regions.

CHAPTER SEVEN

Depth: Two Eyes, One Camera, and the Arithmetic of Distance

A robot arm must pick a part from a bin. The part is a cast bracket, dark grey, lying at an angle among twenty others. The camera knows where the bracket is in the image — a bounding box, a centroid, an orientation — and none of that tells the gripper how far away it is.

The naive answer is to assume a height. The bin has a known depth of 200 millimetres, the parts are 40 millimetres thick, and the top one is probably somewhere in between. This works until it does not, which is on the day the bin is half empty and the top part is 90 millimetres lower than usual. The gripper closes on air, or worse, drives into the part and marks it.

Depth is a separate measurement, and it costs a second camera, a projector, or a time-of-flight sensor. This chapter is the arithmetic that says which one, and how accurate it will be.

Stereo: The Geometry of Disparity

Two cameras, side by side, looking at the same scene. A point in the world projects to a position in the left image and a different position in the right, and the difference between those positions — the **disparity** — encodes distance.

For two cameras with parallel axes, separated by a baseline B, each with focal length f, the depth is

$$Z = f \cdot B / d$$

where d is the disparity in pixels.

Worked. Focal length 8 mm, baseline 120 mm, disparity 4 pixels:

$$Z = 8 \times 120 / 4 = 240 \text{ mm}$$

Now the useful question: how much does a one-pixel error in disparity cost?

$$\text{depth error per pixel} = Z^2 / (f \cdot B)$$

At Z = 240 mm: 57,600 / 960 = 60 mm per pixel. That is enormous — the sensor can localise the point to within 6 cm, which is useless for gripping.

The formula explains the whole behaviour of stereo. The error grows with the square of the distance, because disparity itself shrinks with distance. At one metre, with the same rig, disparity is under one pixel and depth resolution is over a metre per pixel. At 240 mm the system works only if disparity is measured to a fraction of a pixel, which is possible with good texture and careful matching, and impossible on a blank grey casting.

Baseline	Distance	Disparity	Depth error per pixel
120 mm	240 mm	4.00 px	60 mm
120 mm	1,000 mm	0.96 px	1,042 mm
300 mm	1,000 mm	2.40 px	417 mm
300 mm	500 mm	4.80 px	104 mm

Reading the table: to work at a metre you need a baseline of hundreds of millimetres, which makes the rig large and the matching harder, because the two views see genuinely different surfaces. This is the fundamental trade of stereo, and no amount of processing removes it.

Stereo also needs texture. The matching is done by comparing patches, and a patch with no pattern matches everywhere equally well. Projecting a random dot pattern onto the scene — **structured light** — supplies the texture artificially and is one of the great practical tricks of the field.

Structured Light and Laser Triangulation

If you project a known pattern and observe how it deforms, you can compute depth from a single camera. The projection supplies the second "viewpoint", and the pattern supplies the texture.

Two families dominate industry.

Fringe projection. A sequence of sinusoidal patterns is projected while the camera captures each. The phase of the observed sinusoid gives a very precise measurement of position along the projector's axis, and unwrapping across frequencies extends the range. Fringe systems achieve tens of micrometres over

fields of a few hundred millimetres, and they are the standard for dimensional inspection of small parts. They are also slow — several images per measurement — and they dislike moving objects.

Laser line triangulation. A laser projects a line onto the surface; a camera offset by an angle sees the line displaced by an amount proportional to height. Scan the line across the part and you build a profile. The geometry is the same triangle as stereo with one vertex replaced by a projector, and the resolution along the line is set by the camera while the resolution across it is set by the scan step.

Method	Accuracy	Speed	Weakness
Stereo	1–5% of range	Fast	Needs texture; fails on blank surfaces
Fringe projection	0.01–0.1 mm	Slow (0.1–1 s)	Ambient light; shiny parts
Laser line	0.05–0.2 mm	Medium	Speckle; occlusion in deep features
Time of flight	5–20 mm	Fast	Multipath, reflectivity, edges

Time of Flight, and Why It Is Not a Metrology Tool

A time-of-flight camera measures the phase delay of modulated light, and converts it to distance. It is cheap, it produces a full depth image at video rate, and it is excellent for coarse tasks: is there a person in the cell, how full is the bin, is the pallet present.

It is not a metrology tool. Three reasons.

The illumination is modulated at tens of megahertz, and the phase measurement has a resolution of a few millimetres at best. Reflections from nearby surfaces arrive twice — **multipath** — and bias the result towards the nearer surface. And shiny or dark materials return too little light, so the sensor reports depth where there is none and misses depth where there is.

For safety and presence sensing, none of that matters. For measuring a 0.1 mm gap, it is disqualifying.

Calibration: Turning Pixels into Millimetres

Every method above depends on knowing the camera's geometry precisely. **Calibration** estimates two sets of numbers.

Intrinsics describe the camera itself: focal length in pixels, the position of the optical axis, and the lens distortion coefficients. They do not change when the camera is moved.

Extrinsics describe where the camera sits relative to the world, or to the other camera: a rotation and a translation. They change whenever anything is bumped.

The standard procedure is a printed chequered board of known square size, photographed in a dozen or more orientations. A solver finds the parameters that best explain all the views, and reports a **reprojection error**: the average distance, in pixels, between where a corner should appear and where it does.

Reprojection error	Meaning
Under 0.2 px	Excellent; suitable for measurement
0.2–0.5 px	Normal; fine for most tasks
Over 1 px	Something is wrong: motion blur, bad board, wrong square size

Convert the error into physical units before deciding it is good enough. At a sampling interval of 0.28 mm per pixel, 0.3 pixels is 0.08 mm — acceptable for a ±0.2 mm task and not for a ±0.05 mm one.

Two practical points that are learned the hard way. Calibrate at the working distance and with the aperture you will use in production, because focus changes the effective focal length slightly. And re-check calibration after any mechanical work on the rig; a bumped mount is the most common cause of a system that "suddenly" measures 0.3 mm too wide.

What Depth Does Not Solve

Having depth does not mean having understanding. A point cloud of a bin of parts is a surface, and finding the graspable part in it is a separate problem with its own literature.

It also does not solve occlusion. A camera, or a projector, sees one side. Deep recesses, undercuts and the far side of a part are invisible, and no algorithm can measure what was never illuminated. When a drawing specifies a

feature that cannot be seen from any single direction, the answer is a second viewpoint or a different sensor, not better software.

And it does not solve the material. Black anodised aluminium absorbs most of the light; polished steel reflects it away from the sensor; clear plastic transmits it. All three are hard for every method in this chapter, and all three are usually solved by a surface treatment or a different physical principle — a touch probe, an air gauge, or an eddy-current sensor.

Problems

1. A stereo rig has f = 6 mm, B = 200 mm, and resolves disparity to 0.5 pixels. What is the depth error at 300 mm? At 800 mm?

2. You need 2 mm depth accuracy at 500 mm. What baseline do you need with a 6 mm focal length and 0.25-pixel matching? Is that rig mechanically sensible?

3. A calibration reports 0.8 pixels of reprojection error at a sampling interval of 0.1 mm per pixel. What measurement uncertainty does that imply, and what would you check first?

4. Explain, in plain language, why stereo depth gets worse with distance while laser triangulation does not degrade as sharply.

5. A time-of-flight sensor reports a person at 1.2 m and a colleague standing 0.8 m away behind a glass panel. What has probably happened?

6. A bin-picking cell must find the top surface of parts between 300 and 500 mm from the camera, to ±3 mm, in under 100 ms. Which method from the table would you choose, and what would you need to add to make it work on grey castings?

PART THREE

Learning to See

CHAPTER EIGHT

Learning to See

The first version of the scratch detector was four hundred lines of code. Threshold, Sobel, dilate, skeletonise, measure length, filter by orientation, and a set of tuning constants that had been adjusted by hand over three weeks. It worked on the aluminium panels from line one. On line two, where the panels were anodised and slightly darker, it found scratches everywhere.

The replacement was a convolutional network trained on two thousand labelled images from both lines. It took a day to label, an afternoon to train, and it worked on both lines without a tuning constant in sight.

The temptation is to conclude that the network was smarter. What actually happened is more specific and more useful. The hand-written pipeline encoded a *guess* about what a scratch looks like — a thin dark line, roughly straight, longer than 5 millimetres — and the guess was the part that failed. The network did not need the guess. It learned a set of local patterns from examples, and among those patterns were several that the engineer would never have written down: the way a scratch interrupts the fine brushing direction, the way its ends taper, the way a polishing mark differs from a cut.

This chapter explains what a convolutional network is, in the same vocabulary as the previous chapters, and what it is not.

A Layer Is a Stack of Kernels

A single 3 by 3 kernel produces one output image — one filter's opinion about every neighbourhood. A convolutional layer produces dozens: thirty-two kernels, say, each with its own nine weights, applied to the same input.

The output has thirty-two **channels**. Each channel is a map of how strongly that particular pattern occurs at each location. One channel may respond to horizontal edges, another to diagonal texture, another to a specific shade transition. Nobody assigns these roles; they emerge from training.

For colour input there is a further dimension: each kernel is 3 by 3 by 3, with separate weights for red, green and blue, so that the response can depend on colour as well as on shape.

The parameter count follows directly, and it is worth computing because it explains why convolution is efficient:

> parameters per kernel = (3 x 3 x 3) + 1 bias = 28 thirty-two kernels = 896 parameters

A second layer of sixty-four kernels operating on those thirty-two channels needs 3 x 3 x 32 x 64 = 18,432 weights plus 64 biases, or 18,496.

Compare with a fully connected layer on the same image. A 640 by 640 colour image has 1.23 million input values; a single fully connected layer with a thousand outputs would need 1.23 billion weights. That is not merely expensive; it is a structural mistake, because it destroys the spatial arrangement that makes an image an image.

Layer	Input	Weights	Output size
Conv 3x3x32	640x640x3	896	640x640x32
Conv 3x3x64	640x640x32	18,496	640x640x64
Max pool 2x2	640x640x64	0	320x320x64
Dense 1,024 to 10	1,024	10,250	10

Receptive Fields, and Why Depth Beats Width

Each output value of a single 3 by 3 layer depends on a 3 by 3 patch of the input. Stack three such layers and the third depends on a 7 by 7 patch: each step adds two pixels of context in each direction.

> receptive field after n layers of 3 by 3 = 1 + 2n

So four layers see 9 by 9, eight layers see 17 by 17, and twenty layers see 41 by 41. A deep stack of small kernels covers a wide area with far fewer parameters than one large kernel, and — the reason it works — it builds a hierarchy. Early layers see edges, middle layers see combinations of edges, later layers see parts of objects.

This is the same trade already met in Chapter Three with separable filters: structure beats brute force. A single 41 by 41 kernel would need 1,681 weights per channel; twenty 3 by 3 layers need 180.

What the Network Learns, and What It Is Told

Three things determine what a network learns, and only one of them is the architecture.

The labels. If a scratch is labelled only when it is longer than 5 millimetres, the network learns that threshold, faithfully, including its arbitrariness.

The examples. A dataset of panels from one supplier teaches the appearance of that supplier's surface. The network will treat a different surface finish as a different phenomenon, and no amount of confidence will warn you.

The augmentation. This is where the engineer's judgement enters most directly. Augmentation means generating modified copies of the training images — flipped, rotated, brightened, blurred, noised — so the network sees the variations it must tolerate.

It is powerful and it is dangerous, because an augmentation asserts an invariance. Flipping a photo of a scratch is harmless: a scratch looks like a scratch mirrored. Flipping an image of a part with an asymmetric feature is a lie: you have told the network that the mirrored part is the same part, and it will believe you. The test is simple and must be asked every time: *would a human still label this image the same way?* If not, the augmentation is teaching a falsehood.

Transfer Learning, Honestly

Most industrial projects do not train a network from scratch. They start from a model trained on millions of general images and continue training on a few thousand of their own. This works because the early layers of any vision network learn edges, blobs and textures — features that are as true of castings as of cats.

What it buys: usable accuracy with a thousand images instead of a hundred thousand, and faster convergence.

What it costs: a subtle dependence on the pre-training data's assumptions. Networks pre-trained on photographs of scenes expect certain statistics of colour and contrast, and an industrial image — grey, flat, high-contrast, tightly

cropped — is not a photograph of a scene. The mismatch is usually small and occasionally decisive, and the remedy is to fine-tune more layers rather than fewer, so that the later stages can specialise.

A rough rule that holds up in practice: freeze nothing if you have more than a thousand labelled images; freeze the early third if you have a few hundred; and if you have fewer than a hundred, reconsider whether a network is the right tool at all.

What a Network Does Not Do

It does not measure. A network can tell you a scratch is present with high confidence and be unable to tell you it is 6.2 millimetres long to a tenth of a millimetre. Measurement still wants edges, calibration, and geometry.

It does not count reliably. Counting is a global operation, and networks that look at local patches have no mechanism for it; objects that touch or overlap are counted wrongly, confidently.

It does not know when it is out of its depth. A network asked about a surface it has never seen will produce a number, and the number will look like all its other numbers.

It does not replace the light. If the defect is not in the photons, it is not in the network, and the most common cause of a disappointing model is a dataset captured under lighting that never revealed the defect properly in the first place.

Exercises

1. Compute the parameter count for a layer of sixteen 5 by 5 kernels applied to a 640 by 640 single-channel image. How does it compare with sixteen 3 by 3 kernels?

2. Six consecutive 3 by 3 layers: what is the receptive field? How many parameters per channel path, compared with a single 13 by 13 kernel?

3. A dataset contains 1,500 images of steel panels and 200 of aluminium panels. Predict two specific failures of the trained model, and design an augmentation policy that would not fix them.

4. For each augmentation, say whether it is safe, and why: horizontal flip of a scratch image; 90-degree rotation of an image of a gear; brightness change of ±20%; Gaussian blur; colour inversion.

5. A network reports 0.98 confidence on an image of a part type it has never seen. Explain to a plant manager, in three sentences, what that number means.

6. A 640 by 640 image passes through four conv layers of 32, 64, 64 and 128 channels with 3 by 3 kernels, with a 2 by 2 pool after the second and fourth. Sketch the tensor sizes at each stage and compute the multiply-accumulates for the first layer.

CHAPTER NINE

Training a Vision Model Without Fooling Yourself

The report said 99.2%. It was printed, bound, and presented to the steering group, which approved the deployment.

On the line the system found 71% of the defects it was supposed to find, which is still respectable, and 40% of the good parts were rejected, which was not. The plant ran it for a fortnight with every rejection double-checked by hand, and the resulting log — four hundred decisions a day, two weeks — is one of the most useful documents the project produced.

The 99.2% was not a lie. It was an answer to a different question. The dataset had been captured by filming each of forty sample parts for fifty consecutive frames, then splitting the resulting 2,000 images randomly into training and validation. Every part in the validation set had thirty-nine near-identical twins in the training set. The model was not recognising defects; it was recognising parts it had already memorised, and the validation score was measuring that memorisation.

This chapter is about the decisions that make a vision dataset honest, and they are almost entirely about splitting and labelling. Architecture will forgive a great deal. A contaminated split forgives nothing, because it hides the failure until the failure is in production.

The Split Is the Claim

A validation score is a claim about the future. The claim is only as good as the resemblance between the validation data and the future.

There are four ways to split image data, and they answer four different questions.

Split	Construction	Question it answers
Random	Shuffle all images	How well does it fit this dataset?
By object	All images of one part stay together	Can it recognise a new instance?
By scene	All images from one setting stay together	Can it cope with new conditions?
By time	Train on earlier captures, test on later	Will it still work next month?

Random splitting is the default in every library, and it is almost always wrong for images, because images of the same object are not independent samples. Fifty frames of one part are one sample, repeated fifty times, and random splitting distributes that sample across both sides of the wall.

The consequence is arithmetic, not opinion. With 80% of images in training and forty images per part, a validation image has around thirty-nine siblings in training. The model only has to memorise forty appearances to score perfectly. Move to a by-object split and the same model, on a truly unseen part, drops from 99% to somewhere between 75% and 90% — which is the number the line later produced.

Both splits are computable from the same dataset. Only one of them predicted the outcome.

Duplicates Are Everywhere

Frames from a video are the obvious case. Four more are common and less obvious.

The same part photographed twice under slightly different setup. A dataset that looks like 800 images may contain 400 pairs.

Augmented copies stored as separate files. If the augmentation was applied before splitting, every augmented copy of a training image is a near-duplicate, and splitting afterwards puts siblings on both sides. Augment after splitting, never before.

Images from the same physical site. Two cameras on the same line see nearly the same scene; a model tested on the second camera has effectively seen it.

Near-identical backgrounds. A conveyor belt with a distinctive stain appears in a thousand images; the model may learn the stain.

The detection method is embarrassingly simple and almost never done: compute a cheap perceptual hash or a small downsampled fingerprint of every image, sort by similarity, and inspect the closest pairs. Twenty minutes of work, and it converts an unknown into a number.

Augmentation Belongs After the Split

The rule deserves its own paragraph because it is broken so often. Split the originals into training, validation and test. Then augment *only* the training set. Then train. Then evaluate on the unaugmented validation and test images.

If you augment first and split afterwards, the test set contains altered copies of training images, and the score is inflated by exactly the amount the augmentations span. Brightness augmentation, for instance, will make the test set look like training data at every exposure — and the deployed model will then fail on an exposure it has not seen, which is precisely what the test was meant to predict.

Labels, and How Wrong They Are

Human annotation of images disagrees with itself. Published agreement rates for defect labelling typically sit between 85% and 95% depending on the task and the training of the annotators, and your project is not an exception.

Two habits make this visible rather than fatal.

Double-label a sample. Take two hundred images and have a second person label them without seeing the first labels. Report the agreement rate. If it is 88%, then no model can be expected to exceed 88% against the original labels, and a reported accuracy of 99% is a sign of leakage, not excellence.

Write the labelling rules down. "A scratch is any linear mark longer than 2 mm that is visible at the specified lighting" is a rule. Without it, one annotator counts a polish mark as a defect and another does not, and the model learns the disagreement.

The practical consequence is that label noise sets a ceiling. Once a dataset's labels disagree with themselves at 10%, effort is better spent on clearer rules and better images than on a bigger network.

Imbalance Is the Normal Condition

Defects are rare. A dataset with 1.5% positives is typical, and it produces two specific problems.

The model learns to predict the majority class, because that minimises error on the training data. A network that says "good" every time achieves 98.5% accuracy and is worthless.

The metric hides it. Accuracy, as discussed in the previous volume, is dominated by the majority class, and a training log that reports only accuracy will look excellent throughout.

What to do, in order of effectiveness:

Collect more positives. Nothing beats real examples of the thing you must find.

Weight the loss by inverse frequency. A positive example counted fifty times as heavily as a negative one forces attention to the rare class.

Sample the training set deliberately. Balanced batches, with positive examples repeated, work well.

Choose the operating threshold on validation data using costs. This is the step that turns a probability into a decision, and it is separate from training.

Dataset	Positives	Accuracy if always "good"	Useful metric
10,000 images	150 (1.5%)	98.5%	Recall at fixed precision
10,000 images	500 (5%)	95.0%	Precision-recall area
10,000 images	2,000 (20%)	80.0%	Either, with care

Capture Is Engineering

The most valuable hours in a vision project are spent designing the capture, not the model. A coverage matrix is the tool: list the conditions the system must handle, and ensure the dataset contains examples of each.

For a surface inspection cell, the rows might be supplier, surface finish, lighting condition (daylight through the window, lights on, lights off), part orientation, and camera (if more than one). The columns are images needed per cell, with a minimum of perhaps twenty. Filling the matrix takes a week. Discovering an empty cell in production takes a month and a customer complaint.

The matrix also produces the split. If camera is a row, split by camera. If time of day matters, capture across days and split by day. The structure of the split is a statement about what you believe the model must generalise to, and the matrix is the clearest way to say it.

Problems

1. A dataset has 30 parts, 60 frames each, and is split 70/30 randomly. How many frames of a given validation part are likely in training? What does that imply for the measured accuracy?

2. You have 500 images from site A and 200 from site B, and the model must work at both. Describe a split and a reporting scheme that would tell you honestly whether it does.

3. A colleague augments 1,000 images into 8,000 by rotating each in 45-degree steps, then splits randomly. Identify the error and predict its effect on the reported score.

4. Two annotators agree on 87% of 200 images. Estimate the best achievable measured accuracy and explain the reasoning in three sentences.

5. A defect occurs in 0.8% of parts. A model achieves 95% recall and 12% precision. At 5,000 parts per day, how many false alarms per day does that produce, and is it operable?

6. A plant asks whether the model will still work after the summer shutdown, when the ambient temperature rises by eight degrees and the incoming parts come from a second supplier. What would you add to the dataset and to the split to answer that question?

CHAPTER TEN

Detection and Measurement

A defect-detection system flagged the same scratch twice. The operator marked both flags as correct — it was a real scratch — and the part was scrapped. The next day the same thing happened, and the next. When the engineer looked at the log, she found two boxes drawn on one scratch, nine millimetres apart, each with a confidence above 0.9.

The model was not confused. It was a *detector*, trained to put a box around each defect, and it had learned that a long scratch sometimes appears as two shorter segments — because in its training data, some had been labelled that way by annotators who stopped at a change in direction.

Two lessons follow, and this chapter is organised around them. The first is that a detector's output is a set of boxes, and sets require their own arithmetic — overlaps, thresholds, counts. The second is that a detection system which must also *measure* is doing two jobs, and the measurement half needs its own capability study.

Boxes and the IoU

A detection is a rectangle with a class and a confidence: position, size, label, score. To say whether a predicted box is correct, the standard test is **intersection over union**.

```
IoU = area of overlap / area of union
```

Two examples, in pixels.

Box A runs from (10,10) to (50,50), area 40 x 40 = 1,600. Box B runs from (30,30) to (70,70), also 1,600. They overlap from (30,30) to (50,50): 20 x 20 = 400. The union is 1,600 + 1,600 - 400 = 2,800.

```
IoU = 400 / 2,800 = 0.14
```

Two boxes that a person would call the same defect, scoring 0.14, because one is displaced by half its width.

Now shift them closer. A' is (10,10) to (60,60), area 2,500. B' is (20,20) to (70,70), area 2,500. Overlap is (20,20) to (60,60): 1,600. Union is 2,500 + 2,500 - 1,600 = 3,400.

```
IoU = 1,600 / 3,400 = 0.47
```

A conventional match threshold is 0.5, and these two — visibly the same object — fall just short of it. This is why detector evaluation is fussy: the metric is dominated by how tightly the box fits, not by whether the object was found.

For scratches and other elongated objects, IoU behaves even worse, because a small displacement along the long axis barely changes the overlap while a small displacement across it changes everything. The practical answer is to use a metric matched to the shape — distance between centroids, or overlap of a dilated mask — and to state which one you used.

Removing Duplicates

The duplicate in the opening story is the normal output of a detector, not a bug. Sliding-window detectors and their modern descendants produce many overlapping candidates for one object, so a **non-maximum suppression** step runs afterwards.

The procedure: sort candidates by score; take the highest; delete every candidate whose IoU with it exceeds a threshold; repeat.

With a suppression threshold of 0.5, the pair scoring 0.47 above both survive — the duplicate remains. Lower the threshold to 0.3 and one is removed, but genuine pairs of nearby defects may also be merged. With elongated defects the right threshold is often 0.1, and the right IoU is not IoU at all but centroid distance.

Suppression threshold	Duplicates removed	Risk
0.7	Few	Two boxes on one defect
0.5	Most	Occasional duplicates
0.3	Almost all	Nearby defects merged
0.1	All	Distinct defects lost

The choice is a business decision: does a duplicate cost more than a merge? On a scrapping line, a duplicate costs a second look; a merge can hide a second defect. That asymmetry should set the threshold, and it should be written down where the next engineer can find it.

Average Precision, Worked

Detection quality is summarised by precision-recall curves computed per class, then averaged. The arithmetic is short enough to do by hand.

Five detections, two ground-truth defects, sorted by confidence:

Rank	Score	Correct?	Precision	Recall
1	0.95	yes	1.000	0.50
2	0.88	no	0.500	0.50
3	0.80	yes	0.667	1.00
4	0.72	no	0.500	1.00
5	0.60	no	0.400	1.00

Precision is the running fraction of correct detections; recall is the fraction of the two real defects found so far.

The **average precision** is the area under this curve, computed as a sum of rectangles: each time recall increases, the increase is multiplied by the precision at that point and added.

AP = 1.000 x 0.50 + 0.667 x 0.50 = 0.500 + 0.333 = 0.833

A perfect detector scores 1.0. A detector that finds one defect and then emits rubbish scores far less, because the precision collapses before recall completes.

Two cautions. AP is an average over thresholds and says nothing about the operating point you will actually run. And a mean over classes (mAP) can flatter a system that is excellent on a common class and useless on a rare one, since each class contributes equally regardless of how many examples it has. Report per-class numbers for the defects that matter, not only the mean.

Detecting, Then Measuring

A detector answers "is there a defect, and roughly where". A measurement answers "how big is it". Systems that need both should be built in two stages, because the requirements are different.

The first stage finds candidates. It should be tuned generously — a low threshold, accepting false positives — because a missed candidate is unrecoverable, while a false candidate can be rejected by the second stage.

The second stage measures each candidate on the original image, using the tools of Part Two: subpixel edge fitting, calibrated scale, and a stated repeatability.

Calibration converts pixels to millimetres; without it, a length in pixels is a length on the sensor, not on the part. And a stated repeatability is what makes the measurement arguable. "The system reports 6.2 mm" is an assertion. "The system reports 6.2 mm, with a repeatability of ±0.03 mm over thirty repeat images" is an engineering claim.

Capability: Is the Measurement Good Enough?

A measurement is only useful if its uncertainty is small compared with the tolerance it must police. The standard discipline, borrowed from manufacturing metrology, is a **gauge study**: measure the same parts repeatedly, with more than one operator or setup, and decompose the variation.

A worked version. Ten parts, three operators, three trials each, tolerance ±0.25 mm.

Source	Standard deviation
Repeatability (same operator, same part)	0.012 mm
Reproducibility (between operators)	0.008 mm
Combined	$\sqrt{(0.012^2 + 0.008^2)} = 0.0144$ mm

The conventional index is six times the combined deviation divided by the tolerance width:

$$\%GRR = 6 \times 0.0144 / 0.50 = 17\%$$

Under 10% is excellent, 10–30% is acceptable depending on the consequence, and over 30% means the measurement cannot police the tolerance. A

vision system at 17% is a usable gauge. The same system with 0.03 mm repeatability would sit at 36%, and no algorithm change would rescue it — only more light, a better lens, or a larger scale.

This is the most useful number a vision engineer can produce, and it is produced by taking pictures of the same thing many times.

Things to Try

Photograph a coin on a plain background. Draw a box around it by hand, then draw a deliberately sloppy box, and compute the IoU of the two. Then compute the IoU of the sloppy box against a box drawn around a second, slightly different coin. The numbers will show how quickly IoU falls with box quality, and why a threshold of 0.5 is a strong demand.

Then take thirty photographs of the same coin without moving it, measure its diameter in each, and compute the standard deviation. That number, converted to millimetres, is your repeatability — and its relationship to the tolerance you would need is the whole of capability analysis.

CHAPTER ELEVEN

Tracking Across Frames

Two identical bearings travel along a conveyor and cross in front of the camera, one behind the other. In one frame they overlap almost completely. In the next they have separated, and the system — which has been tracking each as a distinct object — assigns the left one's identity to the right-hand position and the right one's to the left.

Downstream, one bearing is recorded as having travelled backwards, and the count for the batch is wrong by two. Nobody notices for a week, because the count is only wrong when parts overlap, and parts only overlap when the line runs unevenly.

Tracking is the discipline of maintaining identity over time, and it is where many otherwise sound vision systems acquire their reputation for being "almost right". This chapter covers the small amount of arithmetic involved, the failure modes that survive it, and the timing budget that decides whether a track is worth anything at all.

Why Track At All

Three reasons, in increasing subtlety.

Counting. A detector that fires on every frame counts one object many times. A track counts once, when the object enters, and is a bookkeeping device.

Speed and direction. The difference between an object's positions in successive frames, divided by the frame interval, gives velocity — which is often the actual quantity of interest, for a conveyor, a roller, or a falling part.

Occlusion. A single frame may not contain the object. A track carries a belief about where it should be, so that a gap of one or two frames is survivable. This is the property that makes tracking indispensable in cluttered scenes.

Association: Who Is Who

The core operation is **association**: given tracks from the previous frame and detections in the current one, decide which detection belongs to which track.

The simplest version uses predicted position. Each track has an estimated position and velocity; extrapolate the position forward by one frame; for each detection, compute the distance to each predicted position; and match the closest pair, subject to a **gate** — a maximum distance beyond which the detection is treated as a new object.

The gate is not arbitrary. It should be set from the motion: an object moving at most 30 pixels per frame with a detection uncertainty of ±5 pixels needs a gate of roughly 3 standard deviations, or 45 pixels.

Worked. A conveyor at 0.2 metres per second, a camera at 30 frames per second, a sampling interval of 0.25 mm per pixel.

> movement per frame = 200 mm/s ÷ 30 = 6.67 mm in pixels = 6.67 / 0.25 = 27 px

A gate of 45 pixels therefore accommodates the expected motion plus four deviations of jitter, and it is about to be tested, because the parts are 40 pixels wide.

The gate is bigger than the part. That is the fundamental difficulty of this example: at this speed and frame rate, an object moves most of its own width between frames, so proximity alone cannot distinguish it from its neighbour. The remedies are to increase the frame rate, reduce the speed, or use a better motion model — and of the three, only the first two are reliable.

One-to-One Assignment

Matching greedily, closest pair first, fails exactly when it matters. Two objects approach each other; the nearest detection to track A is the other object; greedy matching takes it, and track B is left with a detection 60 pixels away, outside the gate, and is dropped.

The correct formulation is an assignment problem: choose the set of pairings that minimises total distance, with each track and each detection used at most once. The classical solution is the Hungarian algorithm, which for realistic numbers of tracks costs nothing measurable. Most vision libraries provide it,

usually under a name like `linear_assignment`.

A practical compromise that works in most cells: compute the full distance matrix, solve the assignment, then discard any pairing whose distance exceeds the gate and treat the remaining unmatched detections as new tracks. This is simple, fast, and — crucially — it cannot produce the crossed assignment that greedy matching produces.

Predict, Then Correct

Once association is done, each track's state is updated. The standard estimator is a **Kalman filter**, whose central idea is small enough to state in one paragraph.

You have two sources of information about where the object is: your prediction from its previous motion, and the measurement in the current frame. Both are uncertain. The filter combines them in proportion to their certainty, giving more weight to whichever is more reliable at that moment.

In one dimension, with a predicted position uncertainty of 3 pixels and a measurement uncertainty of 2 pixels, the weight given to the measurement is

$$\text{gain} = 3^2 / (3^2 + 2^2) = 9 / 13 = 0.69$$

and the estimate becomes 0.69 x measured + 0.31 x predicted. Next frame, the prediction is more certain — it has been corrected — so the gain rises, and the filter trusts its own model more. When a detection is missing, the prediction carries the track for a frame or two, with growing uncertainty, which is exactly the behaviour wanted during an occlusion.

Frames without detection	Track status	Typical action
1	Coasting	Keep, widen gate
2–3	Coasting	Keep, widen further
4–10	Lost	Keep hidden, allow re-acquisition
Over 10	Dead	Remove, count as exited

The numbers are conventions. What matters is that the policy exists and is written down, because the alternative — dropping a track the instant a detection is missed — is why counting systems undercount by a few percent, always in the same direction, and are never quite trusted.

The Cost of an Identity Switch

Identity switches are not cosmetic. Their cost is quantifiable, and quantifying it is the fastest way to justify a faster camera.

Consider a line producing 500 parts per hour, with a tracking system that switches identity once per 100 overlaps, and overlaps occurring for 20% of parts.

```
switches per hour = 500 x 0.20 / 100 = 1
```

One switch per hour sounds trivial, and it is until the consequence is examined: a switch typically produces one double-count and one missing count, so two records are wrong. Over a two-shift day, that is thirty-two wrong records, and if each record is a part that must be traceable, the record system is no longer usable for its intended purpose — which is usually the moment somebody asks for the tracking to be "fixed" by buying a better model, when what the system needs is 60 frames per second instead of 30.

The arithmetic also works in the other direction: at 60 frames per second the movement per frame halves, the gate shrinks below the object width, and the ambiguity that caused the switch disappears. Frame rate is often a tracking parameter, not merely a video parameter.

The Latency Budget

A vision system that reports a position is reporting where the object was, not where it is. If anything downstream acts on that position, the delay between exposure and action must be accounted for, and the accounting is a table.

Stage	Delay
Exposure	2 ms
Sensor readout and transfer	4 ms
Image preprocessing	3 ms
Inference	18 ms
Network and PLC handshake	2 ms
Actuator response	8 ms
Total	37 ms

At 0.2 metres per second the part moves 7.4 millimetres during those 37 milliseconds. If the robot is to grasp a feature that is 6 millimetres wide, the delay is larger than the feature, and the system must compensate: either predict the position at the time of action — which is where the track's velocity estimate earns its keep — or trigger the camera earlier with a photo-eye, so that the part is captured at a known position rather than wherever it happens to be.

The second option is usually better, and it is a hardware decision made in an afternoon. The first is what people attempt when the hardware decision was not made, and it works until the conveyor speed varies.

Exercises

1. A conveyor runs at 0.35 m/s, the camera at 25 frames per second, and the sampling interval is 0.3 mm per pixel. Compute the movement per frame in millimetres and in pixels. Is a gate of 40 pixels sensible for parts 50 pixels wide?

2. A track has a predicted position uncertainty of 5 pixels; a detection has uncertainty of 1.5 pixels. What is the Kalman gain, and what does it mean in words?

3. Two parts cross. Track A is predicted at x = 100, track B at x = 140. Detections appear at x = 118 and x = 122. Explain what greedy matching does and what the correct assignment is.

4. A system counts 12,000 parts per shift and is known to undercount by 1.2%. How many parts are lost per shift, and what three checks would you make first?

5. Total latency is 55 ms and the line speed is 0.5 m/s. How far does a part travel before the actuator responds? What would you change if the tolerance is ±5 mm?

6. Design a tracking policy for a cell in which parts may stop on the conveyor for several seconds. What should a track do when the object has not moved, and what should happen when it moves again?

PART FOUR

Making It Work

CHAPTER TWELVE

Light, Lenses and the Rig

A project was commissioned in November and accepted in February. Between those dates the plant's heating was on, the workshop doors were shut, and the ambient temperature in the inspection cell sat at about 23 degrees. In June the doors opened and the cell reached 33 degrees.

The camera did not care; it was rated to 45. The lens did not care. The part did not care. The *bracket* cared: three hundred millimetres of extruded aluminium, bolted to a frame at one end and carrying the camera at the other, expanded by

```
300 mm x 23 x 10^-6 per °C x 10 °C = 0.069 mm
```

Sixty-nine micrometres. On a task with a tolerance of ±0.1 mm and a sampling interval of 0.04 mm per pixel, that is 1.7 pixels of drift, appearing slowly over a fortnight, with no error message and no obvious cause.

Everything in this chapter is at that level: micrometres, millimetres, and the difference between a rig that measures and a rig that merely takes pictures. It is the least glamorous material in the book and the most decisive.

Choosing a Lens From the Field of View

Three numbers determine the lens, and any two determine the third.

```
focal length = (sensor width x working distance) / field of view width
```

A 8.8 mm sensor, a field of view of 100 mm and a working distance of 400 mm:

```
f = 8.8 x 400 / 100 = 35 mm
```

In practice, choose the working distance from the machine — far enough that the camera is out of the way of hands and tooling — choose the field of

view from the part plus a margin for positioning error, and compute the focal length. Then pick the nearest standard lens and check what the field of view becomes, because lenses come in steps.

The same arithmetic in the other direction is the one that catches people out: given a lens, the working distance required for a field of view may be impossible. A 12 mm lens on that sensor needs a working distance of 136 mm to see 100 mm — closer than most cells allow, and inside the near limit of many lenses.

Depth of Field: The Budget Nobody Makes

A lens focuses at one distance. Either side of it, sharpness falls gradually, and the range over which it is acceptable is the **depth of field**. It shrinks rapidly as magnification rises.

Field of view	Magnification	Depth of field at f/8
300 mm	0.029	about 134 mm
100 mm	0.088	about 16 mm
30 mm	0.293	about 1.7 mm

The numbers assume a blur circle of two pixels and a pixel size of about 3.5 micrometres. Read the bottom row and think about what it implies: at a field of view of 30 mm, only 1.7 mm of the part is sharp. A casting that is 12 mm thick cannot be inspected in one image at that magnification unless the aperture is closed — which costs light, and light costs exposure time, and exposure time costs motion blur.

This chain of trades is the substance of rig design. It is not unusual to discover that a specification is achievable only at f/16 with a five-millisecond exposure, which requires a light four times brighter than the one originally budgeted.

The Diffraction Floor

Closing the aperture has a limit that is not mechanical. Light passing through a small hole spreads, and the spot it forms has a diameter of roughly

2.44 x wavelength x f-number

At f/8 with green light of 0.55 micrometres, that is 10.7 micrometres. At f/16 it is 21.5 micrometres.

Now compare with the pixel: a modern sensor's pixels are around 3.5 micrometres. At f/16 the optical blur covers six pixels, and no amount of resolution in the sensor will recover detail finer than that. This is why small-pixel cameras often perform no better than larger-pixel ones on fine work: the optics, not the sensor, is the limit.

The working compromise for most industrial cells sits between f/4 and f/8, where the lens is sharp, the depth of field is workable at moderate magnification, and diffraction has not yet taken over.

Lighting Is Geometry, Not Brightness

The single most common mistake in machine vision is to treat lighting as a brightness problem. It is a geometry problem: the question is not how much light arrives, but from which directions, and what the surface does with it.

Four arrangements cover most needs.

Bright field. Light arrives from the camera's side, at a shallow angle or through a diffuser above the part. Flat surfaces appear bright, tilted or scratched surfaces appear dark, and texture is emphasised. The standard for surface inspection.

Dark field. Light arrives at a very low angle, nearly parallel to the surface. A flat surface reflects it away from the camera and appears black; any edge, scratch or raised feature scatters light into the camera and appears bright. Superb for finding scratches on polished metal, hopeless for reading a label.

Backlight. The light is behind the part, and the camera sees a silhouette. Contrast is perfect, the part is a black shape on a white field, and every measurement of outline, hole and gap becomes trivial. Useless for surface defects, and the first thing to try whenever a task is about shape.

Dome and coaxial. A dome scatters light from every direction, removing shadows and giving a soft even illumination; coaxial light travels along the lens axis and reflects straight back from shiny flat surfaces, revealing dents as dark spots on a bright field. Both are ways of controlling reflections rather than adding light.

Task	First light to try
Measure the outline of a part	Backlight
Find scratches on polished metal	Dark field, low angle
Read a printed label	Diffuse dome, or coaxial
Detect a dent in painted steel	Coaxial
Segment a dark part from a dark background	Backlight, or a colour that the part reflects

Colour as a Filter

A monochrome camera with a coloured light is often better than a colour camera with white light, because the light chooses what the part reflects before the sensor ever sees it.

The rule is complementary colours. To make a red label disappear into a white background, illuminate with red: the label reflects red strongly, the white background reflects it too — both bright. To make the red label dark against white, illuminate with green or blue: the red label absorbs it, the white background reflects it. One lamp, one decision, and a contrast ratio that no amount of processing could produce.

For metals, wavelength affects penetration and scattering, and the effect can be large — copper under blue light looks different from copper under red. This is an advantage when a material must be distinguished, and a hazard when the ambient light changes colour across a shift.

Motion, Exposure and Light Are One Equation

A conveyor at half a metre per second, an exposure of ten milliseconds, and a sampling interval of 0.1 mm per pixel:

```
smear = 500 mm/s x 0.010 s = 5 mm = 50 pixels
```

Fifty pixels of blur is not a photograph of a part; it is a photograph of a part's journey. To reduce it to one pixel, the exposure must be 0.2 milliseconds — forty times shorter, which requires forty times the light, or a strobe synchronised to the encoder.

This is why high-speed vision is mostly a lighting discipline. Pulsed LEDs delivering the same energy in a hundredth of the time are standard equipment,

and the design question becomes whether the part is in the strobe's field long enough to be illuminated at all.

The Rig Is Part of the Measurement

Everything above assumes the camera stays where it was put. Four mechanisms move it.

Thermal expansion, as in the opening story. Repair: shorter mounts, a material with a lower coefficient, a temperature log, or a reference target visible in every image.

Vibration. A conveyor or a press transmits motion through the floor. A one-millisecond exposure during a vibration of 0.05 mm amplitude produces a blurred edge; a longer exposure produces an average that is stable but biased.

Mechanical creep. Bolted joints settle, particularly in the weeks after installation. This is one reason calibration drifts early and then stabilises.

Dirt. A fingerprint on a lens or a dust speck on a window is a permanent defect in every image. Air purges and scheduled cleaning are not optional, and the cleaning interval should be measured rather than guessed — photograph a white target daily and watch the mean brightness.

The general defence is a **reference target** inside the field of view: a small machined feature, or a fiducial, whose measured position and size are checked every cycle. If the reference moves, the rig has moved, and the correction can be applied to everything else. This single habit converts an unmonitored measurement into a monitored one.

Problems

1. A 11.3 mm sensor must see a 220 mm field of view at 500 mm working distance. What focal length is needed? What field of view would a 25 mm lens give at that distance?

2. A cell needs a depth of field of 8 mm at a field of view of 40 mm. Using the table's trend, is f/8 sufficient? What would you change?

3. Compute the diffraction spot at f/5.6 and compare with a 2.4 micrometre pixel. At what f-number does the spot exceed two pixels?

4. A conveyor runs at 1.2 m/s and the sampling interval is 0.08 mm per pixel. What exposure time keeps the smear under 1.5 pixels?

5. A bracket is 450 mm of steel (coefficient 12 x 10^-6 per °C) and a second is 250 mm of aluminium. Which drifts more over a 15 °C change, and by how much?

6. A task requires distinguishing a black plastic part from a dark grey one on a black conveyor. Propose a lighting arrangement and explain, in terms of reflection, why it works.

7. Design a monitoring scheme that would have detected the thermal drift in the opening story within a week, using equipment already present in the cell.

CHAPTER THIRTEEN

When Vision Is the Wrong Tool

A hole, 6 millimetres in diameter, drilled 22 millimetres deep into a blind bore. The drawing specifies a depth of 22 ±0.15 mm, and the inspector must check every tenth part.

The proposal on the table was a vision system: a camera looking down the bore with a coaxial light, measuring the depth from the apparent position of the bottom face. The quotation was 38,000, and the demonstration worked on a sample.

It was the wrong tool, for a reason that can be stated in one sentence: a camera looking down a 22-millimetre hole sees a circle of light at the bottom, and the relationship between that circle and the depth depends on the reflectivity of the bottom face, the tilt of the drill, the oil film, and the amount of light reaching the bottom — which falls off with the square of the depth. The measurement would work on clean, dry, perpendicular samples and fail on the third shift.

An air gauge cost 9,000, measured depth to 0.01 mm, tolerated oil, and was insensitive to everything except the diameter of the hole. The right answer was not a better camera. It was a different sensor.

This chapter is the discipline of asking that question early.

Four Questions

Before any vision project, ask four questions in order. A "no" at any point is a stop, not a challenge.

Is the information carried by light? If the feature does not change the brightness, colour or polarisation of what leaves the surface, no camera will see it. Depth inside a narrow bore, hardness, internal voids, electrical continuity and material composition are the common cases where the answer is no.

Is the tolerance within reach of the optics? Compute the sampling interval, then the repeatability, then the capability index. A tolerance of ±0.05 mm demands a sampling interval of about 0.01 mm and a repeatability better than 0.01 mm — which is achievable, at a price, on a rigid rig.

Is the environment stable enough? Light, temperature, vibration, dust, coolant spray, and the presence of people all change between the demonstration and the third shift. If the answer is "we will manage it", the project has a hidden cost that will appear later.

Is the cost justified? Not "is it impressive", but "what does a missed defect cost, how many defects occur, and how many years does the capital take to return".

The Alternative Toolkit

Engineers who work only with cameras tend to see every problem as an image. The table is a reminder that the sensing world is wider.

Requirement	Better first choice
Depth in a narrow bore, with oil present	Air gauge
Presence of a metal part at 1 mm	Inductive proximity sensor
Thickness of a sheet, from one side	Ultrasonic gauge
Force, weight, fill level	Load cell
Cracks in conductive material	Eddy current
Sub-micrometre displacement	LVDT or laser interferometer
Internal structure, voids, welds	X-ray or ultrasound
Identification, not inspection	Barcode, RFID
Temperature, stress, gas	Thermal camera, strain gauge, sensor

None of these is exotic. Most cost less than a vision system, and all of them are insensitive to lighting, which is the failure mode that consumes the most engineering time in vision projects.

There is also a class of problem where the correct answer is not a sensor at all but a change to the process: a fixture that makes the feature accessible, a poke-yoke that makes the error impossible, a drill with a depth stop. The best inspection is the one that is not needed.

The Arithmetic of Justification

A vision project is capital spending, and the case is made with numbers.

Suppose a line produces 200,000 parts a year. The current escape rate is 1.2%, meaning 2,400 defective parts reach the customer, each costing 8 in warranty, rework and goodwill: 19,200 a year. A vision system costing 45,000 is predicted to reduce escapes to 0.2%, or 400 parts, costing 3,200.

Item	Before	After
Escapes per year	2,400	400
Cost of escapes	19,200	3,200
Inspection labour	12,000	4,000
Annual saving	—	24,000
Capital	—	45,000
Simple payback	—	1.9 years

The arithmetic is simple and the assumptions are where projects die. Three of them need testing before the capital is committed.

Does the escape rate really fall by that much? A system with 95% recall and 12% precision does not remove 83% of escapes; it removes 95% of the escapes it sees, and it creates a new cost in false alarms. Recompute with the actual operating point, and with the labour cost of handling rejections.

Does the labour saving exist? Inspectors are rarely removed; they are reassigned. The saving is real only if the headcount falls or the capacity is used.

Is the capital complete? The camera, lens, light and enclosure are perhaps half of it. Fixtures, integration, commissioning, validation, training and the first year of support are the rest, and they are the part that surprises.

Where Machine Learning Fits, and Where It Does Not

Inside a vision system, learning is the right tool for appearance problems: texture, scratches, corrosion, weld quality, fabric defects. These are cases where the rule is easy for a person to recognise and hard to write down, which is exactly the signature of a learning problem.

It is the wrong tool for measurement, for counting, for geometric verification, and for anything where the answer must be traceable to a calibrated instru-

ment. A network that reports 6.2 mm has not measured 6.2 mm; it has classified an image into a bin labelled "about 6.2 mm".

The most reliable industrial pattern combines the two: classical geometry where geometry is the question, learning where appearance is the question, and a rule that vetoes physically impossible answers. A model that proposes a 12 mm scratch on a part that is 8 mm wide should be overruled by arithmetic, not by a bigger model.

A Closing Checklist

Ten questions, in the order they should be asked. A project that can answer all ten before writing code has a reasonable chance.

1. What exactly must be decided, in millimetres or in classes, and at what rate?

2. Is the discriminating information in the light, and which lighting geometry reveals it?

3. What sampling interval does the tolerance require, and what field of view does that imply?

4. What is the depth of field, and does the part fit inside it?

5. How fast does the part move, and what exposure does that allow?

6. Where will the light come from in production, and what changes when the sun moves or a lamp ages?

7. What is the repeatability of the measurement, measured over fifty images of one part?

8. What is the capability index against the tolerance, and is it under 30%?

9. How will the dataset be split so that the validation score means something?

10. What is the annual saving, what is the true capital cost, and what is the payback?

Notice that only two of the ten involve software. This is not an accident of the subject; it is the subject.

For Practice

Take a task you know — from work, from a project, from the room you are sitting in — and answer the ten questions for it on one page. Do not look anything up. The gaps in your page are the things to investigate, and the exercise is more valuable than it sounds, because most failed vision projects are failures to write that page.

Then take the four questions and apply them to a problem you have already been told needs vision. If the honest answer is air gauge, load cell or fixture, say so. Saying so early is the most valuable thing a vision engineer does.

Glossary

Aliasing — the appearance of false coarse patterns when a scene is sampled too coarsely. Caused by sampling, not by the software; cured by optics or by moving closer.

Aperture — the adjustable opening in a lens. Smaller openings increase depth of field and reduce light.

Backlight — illumination from behind the part, producing a silhouette. The most reliable lighting for outline measurement.

Bayer pattern — the repeating red-green-blue filter arrangement on a colour sensor. Three quarters of the pixels measure a colour other than the one they are used for.

Bilateral filter — a smoothing filter that preserves edges by weighting neighbours according to both distance and similarity.

Bit depth — the number of levels a sensor can report. Eight bits gives 256; twelve bits gives 4,096.

Blur circle — the diameter of a point image at the sensor. Used to define acceptable focus.

Calibration — the procedure that estimates a camera's internal geometry and its position in the world, from images of a known target.

Capability index (%GRR) — six times the measurement uncertainty divided by the tolerance width. Under 10% is excellent; over 30% is disqualifying.

Canny detector — an edge detector combining smoothing, gradient, non-maximum suppression and hysteresis thresholds.

Connected components — the grouping of touching foreground pixels into labelled regions.

Convolution — the operation of sliding a small kernel across an image, multiplying and summing, to produce a filtered value at every position.

Coaxial light — illumination along the lens axis, revealing flat shiny surfaces and exposing dents as dark spots.

Dark field — illumination at a very low angle, so that flat surfaces appear dark and scratches scatter light into the camera.

Demosaicing — the interpolation that invents the missing colour values at each pixel of a colour sensor.

Depth of field — the range of distances over which the image is acceptably sharp. Falls rapidly as magnification rises.

Diffraction limit — the smallest spot an aperture can form, roughly 2.44 wavelengths times the f-number.

Disparity — the horizontal shift of a point between two stereo images. Inversely proportional to distance.

Erosion — a morphological operation that keeps a foreground pixel only if the whole structuring element fits inside the foreground.

Dilation — the dual operation, growing foreground regions by one element.

Exposure time — the duration over which light is collected. Sets motion blur directly.

Feature (local) — a distinctive image point, such as a corner, described in a way that survives rotation, scale and lighting change.

Field of view — the width and height of the scene that the sensor covers at a given distance.

Fringe projection — a structured-light method using a sequence of sinusoidal patterns to measure depth to high precision.

Gain — amplification applied after collection. Increases signal and noise together.

Gaussian blur — smoothing with weights falling off from the centre. Separable into two one-dimensional passes.

Gradient — the rate of change of brightness, as a vector with horizontal and vertical components.

Homography — the eight-parameter mapping between two views of a flat surface.

Hysteresis — the use of two thresholds so that a weak edge is kept only when it connects to a strong one.

Intersection over union (IoU) — overlap area divided by union area. The standard measure of box agreement.

Intrinsic parameters — the camera's focal length in pixels, optical centre and distortion, independent of where it is mounted.

Kernel — the small grid of weights used in a convolution.

Kalman filter — an estimator that combines a prediction from motion with a measurement, weighted by their uncertainties.

Learning curve — performance plotted against the amount of training data used.

Median filter — a nonlinear filter that replaces each pixel with the median of its neighbourhood; removes impulses without spreading them.

Morphology — operations on binary or grey images defined by a structuring element; opening and closing are the common combinations.

Non-maximum suppression — thinning a response to its local maxima, or removing duplicate detections.

Otsu threshold — the threshold that maximises the variance between the two groups it creates.

Photosite — the individual light-collecting element of a sensor, beneath one colour filter.

Point cloud — a set of three-dimensional points produced by a depth sensor.

Precision and recall — the fraction of positive decisions that are correct, and the fraction of real positives that are found.

RANSAC — a consensus-fitting method that repeatedly samples the minimum number of correspondences and keeps the model with the most agreement.

Receptive field — the region of the input image that influences one output value of a network layer.

Reprojection error — the average pixel distance between where a calibration target's corners should appear and where they do appear.

Repeatability — the spread of repeated measurements of the same object under the same conditions.

Sampling interval — millimetres of scene per pixel. The decisive number for whether a task is possible.

Specular reflection — mirror-like reflection of the light source rather than the surface. Produces highlights that hide detail.

Structured light — projecting a known pattern to supply texture or to encode depth.

Stereo vision — depth from two views, using the geometry of disparity.

Structuring element — the small shape used in morphological operations.

Threshold — the value at which a continuous measurement becomes a decision.

Time of flight — depth from the phase delay of modulated light. Coarse, fast, and robust to texture.

Transfer learning — starting from a network trained on a large general dataset and continuing training on a smaller specific one.

Watershed — segmentation by flooding a brightness landscape from markers.

Sources and Further Reading

The methods in this volume are old, well documented, and mostly public. What follows is a short list of the works worth owning, with a note on what each is good for.

Ballard, D. H. and Brown, C. M., *Computer Vision*, Prentice Hall, 1982. Out of print and still the clearest account of the geometric view of vision. Chapters on perspective, stereo and calibration remain unsurpassed for a reader who wants the reasoning rather than a library call.

Szeliski, R., *Computer Vision: Algorithms and Applications*, second edition, Springer, 2022. The standard modern reference. Free to read online. Use it as a dictionary: it will tell you what a method is called and where the literature goes next.

Hartley, R. and Zisserman, A., *Multiple View Geometry in Computer Vision*, second edition, Cambridge University Press, 2004. The definitive treatment of calibration, homographies and reconstruction. Dense, and worth the effort if your work involves three-dimensional measurement.

Gonzalez, R. C. and Woods, R. E., *Digital Image Processing*, fourth edition, Pearson, 2018. The textbook for filtering, morphology and segmentation, with the arithmetic worked out. The chapters on spatial filtering pair naturally with Chapter Three here.

Nixon, M. and Aguado, A., *Feature Extraction and Image Processing for Computer Vision*, fourth edition, Academic Press, 2019. Practical, engineering-minded, and unusually good on the details that textbooks skip.

Horn, B. K. P., *Robot Vision*, MIT Press, 1986. A short, opinionated book on light, reflectance and the physics of image formation. It explains why the lighting chapters of any vision project are the ones that matter.

Goodfellow, I., Bengio, Y. and Courville, A., *Deep Learning*, MIT Press, 2016. Chapters 9 and 10 cover convolutional networks from first principles. Free online, and still the most careful explanation of what a convolution layer is doing.

Russ, J. C., *The Image Processing Handbook*, seventh edition, CRC Press, 2016. A reference of techniques with pictures. Useful when a specific operation needs a name.

ASTM E2862 and ISO 5725 for measurement capability and repeatability in industrial settings. Dry documents, and the reason a capability figure can be

defended in an argument with a customer.

On the engineering of lighting, the technical notes published by the major illumination manufacturers — Microscan, CCS, Advanced Illumination, and Keyence among them — contain more practical geometry than most textbooks, and they are free.

Two journals are worth scanning for the state of the field: *IEEE Transactions on Pattern Analysis and Machine Intelligence*, and *Journal of Real-Time Image Processing*. Neither will teach a beginner, and both will show where the subject is moving.

Finally, the documentation of the libraries you use. OpenCV's tutorials are uneven and its reference is exhaustive. Read the reference for the function you are calling at least once, particularly the parts about coordinate conventions, and you will avoid the class of error that Chapter Two describes.